

CONHECIMENTOS ESPECÍFICOS

Linguagem Javascript



Presidente: Gabriel Granjeiro

Vice-Presidente: Rodrigo Calado

Diretor Pedagógico: Erico Teixeira

Diretora de Produção Educacional: Vivian Higashi

Gerente de Produção Digital: Bárbara Guerra

Coordenadora Pedagógica: Élica Lopes

Todo o material desta apostila (incluindo textos e imagens) está protegido por direitos autorais do Gran. Será proibida toda forma de plágio, cópia, reprodução ou qualquer outra forma de uso, não autorizada expressamente, seja ela onerosa ou não, sujeitando-se o transgressor às penalidades previstas civil e criminalmente.

CÓDIGO:

250205554023



PATRÍCIA QUINTÃO

Mestre em Engenharia de Sistemas e computação pela COPPE/UFRJ, Especialista em Gerência de Informática e Bacharel em Informática pela UFV. Atualmente é professora no Gran Cursos Online; Analista Legislativo (Área de Governança de TI), na Assembleia Legislativa de MG; Escritora e Personal & Professional Coach. Atua como professora de Cursinhos e Faculdades, na área de Tecnologia da Informação, desde 2008. É membro: da Sociedade Brasileira de Coaching, do PMI, da ISACA, da Comissão de Estudo de Técnicas de Segurança (CE-21:027.00) da ABNT, responsável pela elaboração das normas brasileiras sobre gestão da Segurança da Informação. Autora dos livros: Informática FCC - Questões comentadas e organizadas por assunto, 3ª. edição e 1001 questões comentadas de informática (Cespe/UnB), 2ª. edição, pela Editora Gen/Método. Foi aprovada nos seguintes concursos: Analista Legislativo, na especialidade de Administração de Rede, na Assembleia Legislativa do Estado de MG; Professora titular do Departamento de Ciência da Computação do Instituto Federal de Educação, Ciência e Tecnologia; Professora substituta do DCC da UFJF; Analista de TI/Suporte, PRODABEL; Analista do Ministério Público MG; Analista de Sistemas, DATAPREV, Segurança da Informação; Analista de Sistemas, INFRAERO; Analista - TIC, PRODEMGE; Analista de Sistemas, Prefeitura de Juiz de Fora; Analista de Sistemas, SERPRO; Analista Judiciário (Informática), TRF 2ª Região RJ/ES, etc.

GRAN
CONCURSOS

SUMÁRIO

Apresentação	5
Linguagem Javascript.....	6
Termos, Definições e Abreviaturas.....	6
Javascript: Introdução.....	7
Histórico	7
Javascript e Tipagem.....	9
Principais Estruturas da Linguagem.....	14
Operações Assíncronas em JavaScript	41
JavaScript no Lado Cliente (Web Browsers)	46
Fetch API	46
XML HTTP Request.....	47
HTML Básico	48
Tags HTML	49
DOM	57
Principais Frameworks Javascript no Cliente.....	58
JavaScript no Servidor.....	59
NodeJs.....	59
Ferramentas do Ecossistema JavaScript e suas Utilidades	61
Babel	61
Webpack	62
ESLint.....	62
Prettier	63
Typescript	63
Vite.....	64
Jest	64
Resumo	66

Questões de Concurso.....	70
Gabarito	85
Gabarito Comentado.....	86
Referências	115

APRESENTAÇÃO

Olá, querido(a) amigo(a), tudo bem? 😊

Seja muito bem-vindo(a) a esta aula sobre **a linguagem JavaScript!** Lembre-se:

O futuro recompensa aqueles que seguem em frente. Eu não tenho tempo para sentir pena de mim mesmo. Eu não tenho tempo para reclamar. Eu vou seguir em frente. (Steve Jobs).

Espero que você aproveite ao máximo cada conceito e cada linha de código que vamos explorar juntos. Continue firme nessa caminhada e tenha ótimos estudos!

Um abraço,

Profª Patrícia Lima Quintão 🚀 ✨



LINGUAGEM JAVASCRIPT

TERMOS, DEFINIÇÕES E ABREVIATURAS

A seguir destacamos alguns **termos**/definições e abreviaturas:

Variável: um nome simbólico para um valor que pode ser armazenado e manipulado em um programa.

Função: um bloco de código reutilizável que executa uma tarefa específica.

Objeto: uma coleção de propriedades, em que cada propriedade é uma associação entre um nome (ou chave) e um valor.

Array: uma estrutura de dados que armazena uma coleção ordenada de elementos.

String: uma sequência de caracteres usada para representar texto.

Number: um tipo de dado que representa valores numéricos, incluindo inteiros e números de ponto flutuante.

Boolean: um tipo de dado que representa um valor verdadeiro (true) ou falso (false).

Undefined: um valor que indica que uma variável foi declarada, mas ainda não recebeu um valor.

Null: um valor que representa a ausência intencional de qualquer valor ou objeto.

Operador: um símbolo que realiza uma operação em um ou mais operandos (valores ou variáveis).

Expressão: uma combinação de valores, variáveis e operadores que resulta em um único valor.

Condicional: uma instrução que executa diferentes blocos de código com base em uma condição (if, else).

Loop: uma estrutura de controle que repete um bloco de código várias vezes (for, while).

DOM (Document Object Model): uma interface que permite que scripts acessem e manipulem o conteúdo, a estrutura e o estilo de documentos HTML e XML.

Evento: uma ação ou ocorrência que acontece no navegador, como um clique do mouse ou o carregamento de uma página.

Callback: uma função passada como argumento para outra função, que é executada quando um determinado evento acontece ou uma tarefa é concluída.

Closure: uma função que tem acesso ao escopo de outra função, mesmo após a função externa ter terminado de executar.

Hoisting: um comportamento em JavaScript em que declarações de variáveis e funções são movidas para o topo de seu escopo antes da execução do código.

AJAX (Asynchronous JavaScript and XML): uma técnica que permite que páginas web atualizem conteúdo de forma assíncrona, sem a necessidade de recarregar a página inteira.

JSON (JavaScript Object Notation): um formato leve para troca de dados, baseado em um subconjunto da sintaxe de JavaScript, usado para representar dados estruturados.

JS: Javascript.

JAVASCRIPT: INTRODUÇÃO

JavaScript é uma linguagem de *script* orientada a objetos, multiplataforma, projetada para adicionar interatividade às páginas web, como animações elaboradas, botões clicáveis e menus suspensos. Além de seu uso tradicional **no lado cliente**, versões avançadas **no lado servidor**, como o Node.js, expandem suas capacidades, permitindo funcionalidades que vão além do simples *download* de arquivos — por exemplo, possibilitando colaboração em tempo real entre múltiplos dispositivos. Em um ambiente de hospedagem, como um navegador web, o JavaScript interage com os objetos desse ambiente, oferecendo controle programático sobre eles.

A linguagem possui uma biblioteca padrão que inclui objetos como Array, Date e Math, além de elementos fundamentais, como operações, estruturas de controle e declarações. O núcleo do JavaScript é altamente extensível, permitindo adaptações para diferentes finalidades:

- **JavaScript no lado cliente:** amplia o núcleo da linguagem com objetos que controlam o navegador e o Document Object Model (DOM). Isso permite, por exemplo, que uma aplicação manipule elementos em formulários HTML, responda a eventos do usuário (como cliques ou entradas de texto) e gerencie a navegação na página.
- **JavaScript no lado do servidor:** estende o núcleo com objetos específicos **para execução em servidores**. Com isso, uma aplicação pode se comunicar com bancos de dados, manter a continuidade de informações entre requisições ou realizar operações em arquivos no servidor.

Assim, no navegador, o JavaScript pode transformar dinamicamente a estrutura e o estilo de uma página web **por meio do DOM**. Já no servidor, com **Node.js**, ele é capaz de processar requisições personalizadas enviadas pelo código cliente, oferecendo uma integração robusta entre *front-end* e *back-end*.

HISTÓRICO

O JavaScript tem uma história que reflete tanto a evolução da web quanto as demandas por interatividade *online*.

JavaScript foi criado em 1995 por Brendan Eich, então engenheiro da Netscape Communications. **A linguagem surgiu como uma resposta à necessidade de tornar as páginas web mais dinâmicas**, que até então eram majoritariamente estáticas, compostas

apenas por HTML e CSS. Inicialmente chamada de Mocha, foi desenvolvida em apenas 10 dias durante o trabalho de Eich no Netscape Navigator 2.0. Logo após, foi renomeado para LiveScript, mas, em uma estratégia de *marketing* para aproveitar a popularidade do **Java** (uma linguagem da Sun Microsystems), acabou recebendo o nome definitivo de JavaScript, apesar de não ter relação técnica com Java.

O objetivo inicial era simples: **permitir scripts leves que rodassem no lado do cliente (navegadores), manipulando elementos da página e respondendo a ações do usuário, como cliques e formulários**. A primeira versão era limitada, mas já incluía conceitos como funções e objetos.

Com o rápido crescimento da web, a Netscape decidiu submeter o JavaScript à ECMA International (*European Computer Manufacturers Association*) para padronização, garantindo consistência entre diferentes navegadores. Em 1997, foi lançado o ECMAScript 1 (ES1), o padrão formal que define a linguagem. O nome “ECMAScript” foi escolhido para evitar disputas de marca, mas “JavaScript” continua sendo o nome comercial usado pela comunidade. A Microsoft, por sua vez, lançou uma versão simultânea chamada JScript no Internet Explorer 3.0, o que gerou as famosas “guerras dos navegadores” e desafios de compatibilidade.

Nos anos seguintes, o ECMAScript evoluiu lentamente:

- 1998 – ES2: pequenas correções.
- 1999 – ES3: introduziu recursos importantes como expressões regulares, *try-catch* e melhorias na manipulação de *strings* e *arrays*. Tornou-se uma base de JavaScript por quase uma década.

Durante esse período, o JavaScript ganhou relevância com a ascensão de bibliotecas como Prototype e jQuery, que facilitaram a manipulação do DOM (*Document Object Model*) e contornaram diferenças entre navegadores.

Entre 2005 e 2015, o JavaScript experimentou um renascimento com o advento da Web 2.0, que priorizou aplicações ricas e interativas:

- 2005: Jesse James Garrett conhece o termo AJAX (Asynchronous JavaScript and XML), destacando o uso do JavaScript para requisições assíncronas, como Gmail e Google Maps.
- 2009 – ES5: trouxe melhorias significativas, como métodos de array (*forEach*, *map*, *filter*), suporte a JSON nativo (*JSON.parse* e *JSON.stringify*) e modo estrito (“*use strict*”).

A publicação do ECMAScript 2015 (ES6) em 2015 marcou uma revolução, **modernizando a linguagem** com:

- Funções de seta (arrow functions) (*=>*).
- Declarações *let* e *const*.

- Promessas para assincronicidade. ATENÇÃO!!! A assincronicidade SEMPRE foi uma característica do JS, ela só foi melhorada com a inclusão das promessas
- Módulos (importação e exportação).
- Desde então, o ECMAScript passou a ter atualizações anuais:
- ES2016 (ES7): Operador de exponenciação (******) e Array.includes.
- ES2020 (ES11): BigInt, Encadeamento Opcional (**?.**), Coalescência Nula (**??**).
- Expansão para o Servidor: Node.js (2009)

Em 2009, Ryan Dahl lançou o Node.js, baseado no motor V8 do Google Chrome, **permitindo que o JavaScript fosse executado fora do navegador, no lado do servidor**. Isso transformou o JS em uma **linguagem full-stack, usada para APIs, servidores web e até ferramentas de linha de comando**.

Hoje JavaScript é uma das linguagens mais populares do mundo, segundo índices como o Stack Overflow e o GitHub. É essencial na web moderna, impulsionado por *frameworks* como React, Angular e Vue.js, e continua evoluindo com o suporte da comunidade e da ECMA. **Suas características — do cliente ao servidor — e a vasta gama de bibliotecas e ferramentas (como TypeScript e Webpack)** consolidam seu papel como pilar da tecnologia atual (MDN – *Javascript Guide*).

JAVASCRIPT E TIPAGEM

JavaScript é uma **linguagem de programação dinâmica e versátil**, extremamente especializada por sua flexibilidade na manipulação de tipos de dados.

Diferentemente de linguagens como Java ou C++, que possuem tipagem estática e desativam a declaração explícita de tipos para variações, **o JavaScript adota uma abordagem de tipagem dinâmica e fraca**. Isso significa que os tipos de variáveis são determinados em tempo de execução e que a linguagem realiza ações automáticas entre tipos em determinadas situações, um processo conhecido como **coerção**.

Nesta seção, exploraremos detalhadamente como o JavaScript lida com tipos de dados, o operador ***typeof*** e as implicações práticas dessa abordagem.

Tipagem Dinâmica:

A tipagem dinâmica do JavaScript permite que uma variável mude de tipo ao longo da execução do código sem a necessidade de declarações prévias. Em vez de fixar um tipo ao criar uma variável, o interpretador JS atribui o tipo com base no valor que ela recebe.

Veja o exemplo:

```
let variavel = 42;           // 'variavel' é um número (number)
console.log(typeof variavel); // "number"

variavel = "Olá, mundo!"; // Agora é uma string
console.log(typeof variavel); // "string"

variavel = true;           // Agora é um booleano
console.log(typeof variavel); // "boolean"
```

No exemplo anterior, uma variável assume diferentes tipos (*number*, *string*, *boolean*) sem gerar erros, demonstrando a flexibilidade da **tipagem dinâmica**. Isso é útil para um desenvolvimento rápido, mas exige cuidado para evitar comportamentos inesperados.

Tipagem Fraca e Coerção

Além de ser dinâmico, o JavaScript é uma linguagem de tipagem fraca, o que significa que ele realiza ações automáticas (coerção) ocultas entre tipos para facilitar as operações.

A coerção pode ser implícita (feita pelo interpretador) ou explícita (controlada pelo programador). Veja exemplos:

- **Coerção Implícita (feita pelo navegador):**

```
let numero = 5;
let texto = "10";
let resultado = numero + texto; // Coerção: número é convertido para string
console.log(resultado); // "510" (concatenação, não soma)

console.log(numero * texto); // Coerção: string "10" vira número
// 50 (multiplicação numérica)
```

Nesse primeiro caso, o operador **+** força a conversão de número para *string*, resultando em **concatenação**. No segundo, o operador ***** converte texto para número, permitindo a **multiplicação**. Esse comportamento é típico de tipagem fraca e pode gerar confusão se não for bem compreendido.

- **Coerção Explícita (controlada pelo programador)**

O programador pode forçar as funções adicionais usando funções como `Number()`, `String()` ou `Boolean()`:

```
let texto = "123.45";
let numero = Number(texto); // Converte string para número
console.log(numero); // 123.45

let valor = 0;
console.log(Boolean(valor)); // false (0 é falsy)
```

Tipos de Dados em JavaScript

JavaScript possui dois grandes grupos de tipos: **primitivos e não primitivos (objetos)**.

Vamos detalhar cada um.

• Tipos Primitivos

Os tipos primitivos são imutáveis e representam valores básicos:

- **número**: representa números inteiros e de ponto flutuante (ex.: 42, 3.14).

```
console.log(typeof 42); // "number"
console.log(typeof NaN); // "number" (Not a Number é um caso especial)
```

- **string**: sequências de caracteres (ex.: "texto", 'outra string').

```
console.log(typeof "Olá"); // "string"
```

- **booleano**: valores lógicos verdadeiro ou falso.

```
console.log(typeof true); // "boolean"
```

- **undefined**: indica que uma variável foi declarada, mas não inicializada.

```
let x;
console.log(typeof x); // "undefined"
```

- **null**: representa a ausência intencional de valor.

```
let y = null;
console.log(typeof y); // "object" (um bug histórico do JS)
```

- **símbolo**: introduzido no ES6, usado para criar identificadores únicos.

```
const id = Symbol("id");
console.log(typeof id); // "symbol"
```

- **bigint**: Introduzido no ES2020, para números inteiros muito grandes (ex.: 123n).

```
const grande = 123n;
console.log(typeof grande); // "bigint"
```

• Tipos Não Primitivos (Objetos)

Os objetos são coleções de propriedades e métodos, incluindo:

- **Objeto**: estrutura chave-valor

```
const pessoa = { nome: "Ana", idade: 25 };
console.log(typeof pessoa); // "object"
```

- **Array**: lista ordenada de valores.

```
const lista = [1, 2, 3];
console.log(typeof lista); // "object"
```

- **Função:** em Javascript uma função também é um tipo.

```
const soma = function(a, b) { return a + b; };
console.log(typeof soma); // "function" (subtipo de object)
```

O Operador typeof

O **operador typeof** é a principal ferramenta para verificar o tipo de uma variável em tempo de execução. Ele retorna uma *string* com o nome do tipo:

```
console.log(typeof 42);           // "number"
console.log(typeof "texto");      // "string"
console.log(typeof true);         // "boolean"
console.log(typeof undefined);    // "undefined"
console.log(typeof null);         // "object" (bug)
console.log(typeof {});           // "object"
console.log(typeof []);           // "object"
console.log(typeof function(){}); // "function"
```

- Limitações do **typeof**
 - **Não diferencia subtipos de objetos** (ex.: Array e Object retornam "object").
 - O caso de nulo é inconsistente.

Para verificações mais precisas desses tipos, use métodos como `Array.isArray()` ou `instanceof`.

Diferença entre === e ==

JavaScript oferece dois operadores de igualdade, que são:

== (igualdade solta); e

=== (igualdade estrita).

A diferença está na coerção de tipos:

- **== (Igualdade solta):** compara valores após coerção implícita, ou seja, tenta converter os operandos para o mesmo tipo antes de comparar.

```
console.log(5 == "5"); // true (string "5" é convertida para número)
console.log(0 == false); // true (false é convertido para 0)
console.log(null == undefined); // true (caso especial)
```

- **=== (Igualdade Estrita):** compara valores e tipos sem coerção. Só retorna true se ambos forem iguais em valor e tipo.

```
console.log(5 === "5"); // false (number ≠ string)
console.log(0 === false); // false (number ≠ boolean)
console.log(null === undefined); // false (tipos diferentes)
```

Recomendação: Use **===** para evitar surpresas com coerção, garantindo comparações mais previsíveis.

Diferença entre `!=` e `!==`

Da mesma forma, os **operadores de desigualdade têm comportamentos distintos**:

- **`!=` (Desigualdade Solta)**: verifica se os valores são diferentes após coerção implícita.

```
console.log(5 != "5"); // false ("5" vira 5, igual)
console.log(0 != false); // false (false vira 0, igual)
console.log(null != undefined); // false (caso especial)
```

- **`!==` (Desigualdade Estrita)**: verifica se os valores ou tipos são diferentes, sem coerção.

```
console.log(5 !== "5"); // true (number ≠ string)
console.log(0 !== false); // true (number ≠ boolean)
console.log(null !== undefined); // true (tipos diferentes)
```

Valores Verdade e Falsidade

JavaScript avalia expressões em contextos *booleanos*, classificando valores como *true* (verdadeiros) ou *false* (falsos):

- **Falsy**: false, 0, -0, "" (string vazia), null, indefinido, NaN.
- **Truthy**: Todos os outros valores, como "texto", 1, [], {}.

```
if (0) {
  console.log("Verdadeiro"); // Não executa
} else {
  console.log("Falso"); // Executa
}

if ("texto") {
  console.log("Verdadeiro"); // Executa
}
```

Implicações Práticas

A tipagem dinâmica e falha do JavaScript oferece vantagens e desafios:

- **vantagens**: flexibilidade e rapidez no desenvolvimento, ideal para prototipagem.
- **desafios**:
 - erros sutis devido à coerção (ex.: "2" + 2 resultado em "22");
 - dificuldade em prever tipos em códigos complexos.

Para mitigar esses problemas, boas práticas incluem:

- Usar comparações estritas (`===` e `!==`) para evitar coerção.
- Documentar tipos esperados em funções.
- Adotar ferramentas como TypeScript para tipagem estática opcional.

Exemplo Prático

Considere uma função que soma valores:

```
function somar(a, b) {
    return a + b;
}

console.log(somar(2, 3)); // 5
console.log(somar("2", 3)); // "23" (coerção)
console.log(somar("2", "3")); // "23" (concatenação)

function somarNumeros(a, b) {
    return Number(a) + Number(b); // Coerção explícita
}

console.log(somarNumeros("2", "3")); // 5
```

Esse exemplo ilustra como a tipagem fraca pode alterar os resultados e como controlá-la.

PRINCIPAIS ESTRUTURAS DA LINGUAGEM

As principais estruturas de uma linguagem de programação são os elementos fundamentais que permitem a construção de programas funcionais e organizados. Elas incluem variáveis e tipos de dados, que armazenam informações; operadores, que realizam cálculos e comparações; estruturas de controle, como condicionais e *loops*, que definem o fluxo de execução; funções, que agrupam blocos de código reutilizáveis; e estruturas de dados, como arrays e listas, que organizam informações de maneira eficiente. Essas estruturas são a base para o desenvolvimento de qualquer software, garantindo clareza, eficiência e manutenção do código.

Definição de variáveis **var**, **let** e **const**

a) **var**

- **Definição:** a palavra-chave **var** foi a primeira forma de declaração de variáveis em JavaScript, modificada desde suas versões iniciais (ES5 e anteriores).
- **Escopo:** o **var** tem escopo global ou de função. Isso significa que uma variável declarada com **var** é válida em todo o código (se declarada fora de uma função) ou apenas dentro da função onde foi declarada.
- **Reatribuição:** permite alterar o valor da variável após a declaração.
- **Hoisting:** variáveis declaradas com **var** são “elevadas” (**hoisted**) ao topo do seu escopo, mas apenas a declaração, não a inicialização. Isso pode gerar comportamentos inesperados.
- **Problemas:** por causa do escopo amplo e do içamento, o uso de **var** pode levar a erros difíceis de rastrear, especialmente em códigos maiores.

- **Exemplo de var:**

```
console.log(x); // undefined (devido ao hoisting)
var x = 10;
console.log(x); // 10

function exemploVar() {
  var y = 20;
  if (true) {
    var y = 30; // Sobrescreve a variável y da função inteira
    console.log(y); // 30
  }
  console.log(y); // 30 (não há escopo de bloco!)
}
exemploVar();
```

Seguem dois exemplos de hoisted, um para funções e outro para variáveis

```
1 greet(); // Repare que estamos chamando greet antes de declarar, isso ocorre pois a função foi "elevada" ao início do escopo
2 function greet() {
3   console.log("Olá função hoisted!");
4 }
5
```

```
1 console.log(myVar); // Já estamos utilizando a variável antes da declaração, como ela é um var, é elevada ao início do escopo
2 var myVar = "Olá, Hoisting!";
3 console.log(myVar); // Vai retornar o mesmo valor da linha um
4
```

b) let

- **Definição:** introduzida no ES6 (2015), é uma alternativa moderna ao var.
- **Escopo:** o let tem escopo de bloco. Isso significa que uma variável só é válida dentro do bloco onde foi declarada (delimitada por {}), como em loops ou condicionais.
- **Reatribuição:** permite alterar o valor da variável após a declaração.
- **Hoisting:** assim como var, sofre hoisting, mas não pode ser acessado antes da declaração (gera um erro chamado "Temporal Dead Zone" – TDZ).
- **Vantagens:** mais previsível e seguro que var, pois respeita o escopo do bloco.
- **Exemplo de let:**

```
const PI = 3.14;
// PI = 3.1415; // TypeError: Assignment to constant variable
console.log(PI); // 3.14

function exemploConst() {
  const lista = [1, 2, 3];
  lista.push(4); // Modifica o conteúdo (permitido)
  console.log(lista); // [1, 2, 3, 4]
  // lista = [5, 6, 7]; // TypeError: Assignment to constant variable
}
exemploConst();
```

c) constante

- **Definição:** também introduzido no ES6, const é usado para declarar variáveis que não terão seu valor reatribuído.
- **Escopo:** assim como let, tem escopo de bloco.
- **Reatribuição:** não permite reatribuição do valor após a declaração. Porém, se o valor para um objeto ou array, as propriedades ou elementos podem ser alterados.
- **Hoisting:** sofre hoisting, mas também está sujeito à Zona Morta Temporal (TDZ), como o let.
- **Uso:** ideal para valores fixos, como constantes matemáticas ou configurações que não mudam.
- **Exemplo de const:**

```
const PI = 3.14;
// PI = 3.1415; // TypeError: Assignment to constant variable
console.log(PI); // 3.14

function exemploConst() {
  const lista = [1, 2, 3];
  lista.push(4); // Modifica o conteúdo (permitido)
  console.log(lista); // [1, 2, 3, 4]
  // lista = [5, 6, 7]; // TypeError: Assignment to constant variable
}
exemploConst();
```


d) Diferenças entre var, let e const

Característica	var	let	const
Escopo	Global ou de função	Bloco	Bloco
Reatribuição	Sim	Sim	Não (mas objetos podem ser modificados)
Hoisting	Sim (undefined)	Sim (TDZ)	Sim (TDZ)
Uso recomendado	Evitar (legado)	Uso geral	Valores fixos

e) Em quais escopos var e let são válidos?

O **escopo** define a região do código em que uma variável é acessível. Vamos detalhar os tipos de escopo em JavaScript:

- **Escopo global:** variáveis declaradas fora de qualquer função ou bloco são globais e acessíveis em todo o código.

```
var globalVar = "Eu sou global";
console.log(globalVar); // "Eu sou global"
```

- **Escopo de função:** variáveis declaradas com var dentro de uma função só são acessíveis dentro dela.

```
function teste() {
  var localVar = "Eu sou local";
  console.log(localVar); // "Eu sou local"
}
// console.log(localVar); // ReferenceError: localVar is not defined
```

- **Escopo de bloco:** variáveis declaradas com let ou const dentro de um bloco {} (como em if, for, etc.) só existem naquele bloco.

```
if (true) {
  let blocoVar = "Eu sou de bloco";
  console.log(blocoVar); // "Eu sou de bloco"
}
// console.log(blocoVar); // ReferenceError: blocoVar is not defined
```

Exemplo comparativo:

```
function escopoComparado() {
  if (true) {
    var varTeste = "var aqui";
    let letTeste = "let aqui";
  }
  console.log(varTeste); // "var aqui" (escopo de função)
  // console.log(letTeste); // ReferenceError (escopo de bloco)
}
escopoComparado();
```

f) Exercício prático

Escreva um código que:

1. Declare uma const com o valor de um raio (ex.: 5).
2. Use let para calcular a área de um círculo ($\pi * \text{raio}^2$).
3. Use var dentro de uma função para demonstrar o escopo da função.

Solução:

```
const RAI0 = 5;
let area = 3.14 * RAI0 * RAI0;
console.log("Área:", area); // Área: 78.5

function mostrarEscopo() {
  var mensagem = "Eu sou visível só aqui";
  console.log(mensagem);
}
mostrarEscopo();
// console.log(mensagem); // ReferenceError
```

Comentários

Os **comentários** são uma parte essencial de qualquer linguagem de programação, incluindo JavaScript. Eles **permitem que os desenvolvedores adicionem anotações, explicações ou desativem trechos de código sem que isso afete a execução do programa.**

Em JavaScript, os comentários não são interpretados pelo motor da linguagem, ou seja, são ignorados durante a execução.

Eles servem para melhorar a legibilidade do código, documentar funcionalidades e facilitar o trabalho entre programadores.

Em Javascript existem dois tipos principais de comentários: comentários de uma linha e comentários de várias linhas. Vamos explorar cada um deles em detalhes, com exemplos e casos de uso.

a) Comentários de Uma Linha

- **Sintaxe:** usa duas barras (//) no início da linha. Tudo após as barras, até o final da linha, é considerado um comentário.
- **Uso:** ideal para explicações curtas, anotações rápidas ou para desativar uma única linha de código.
- **Características:** o comentário termina automaticamente ao final da linha, não sendo necessário fechá-lo.
- **Exemplo Básico:**

```
// Declara uma variável para armazenar o nome
let nome = "João";
console.log(nome); // Exibe "João" no console
```

- **Exemplo com desativação de código:**

```
let idade = 25;
// let idade = 30; // Linha desativada, não será executada
console.log(idade); // 25
```

b) Comentários de Várias Linhas

- **Sintaxe:** inicia com /* e termina com */. Todo o conteúdo entre esses marcadores é considerado um comentário, mesmo que ocupe várias linhas.
- **Uso:** perfeito para explicação longa, documentação detalhada ou para desativar blocos inteiros de código.
- **Características:** pode abranger várias linhas e deve ser explicitamente fechado com */. Não pode haver outro /* dentro de um comentário já aberto (não suporta aninhamento nativo).
- **Exemplo Básico:**

```
/*
    Esta função soma dois números e retorna o resultado.
    Ela será usada para cálculos simples no programa.
*/
function somar(a, b) {
    return a + b;
}
console.log(somar(3, 4)); // 7
```

- **Exemplo com Desativação de Bloco:**

```
let x = 10;
/*
let y = 20;
let z = x + y;
console.log(z);
*/
console.log(x); // Apenas 10 é exibido
```

c) Usos comuns de comentários

Comentários têm diversas questões específicas em JavaScript. Aqui estão os usos mais frequentes:

- **Documentação:** explicar o propósito de uma função, variável ou bloco de código.

```
// Função que converte temperatura de Celsius para Fahrenheit
function converterParaFahrenheit(celsius) {
    return (celsius * 9/5) + 32;
}
```

- **Depuração:** desativar temporariamente partes do código para testes.

```
let resultado = 100;
// console.log("Teste:", resultado * 2); // Linha desativada para análise
console.log(resultado); // 100
```

- **Organização:** separar cláusulas do código ou adicionar lembretes.

```
// === Configurações iniciais ===
const TAXA = 0.1;
let saldo = 1000;

// === Cálculos ===
let juros = saldo * TAXA;
```

- **Anotações Legais ou de Autoria:** comentário usado em arquivos maiores.

```
/*
 * Projeto: Calculadora Simples
 * Autor: João Silva
 * Data: 23/03/2025
 */
```

d) Limitações e Curiosidades

- **Não Executável:** comentários não afetam o desempenho do código, mas aumentam o tamanho do arquivo. Ferramentas como **minificadores** removem comentários em versões de produção.
- **Erros de Sintaxe:** um `*/` sem um `/*` correspondente gera erro.

```
let x = 10;
*/ // SyntaxError: Unexpected token
```

- **Comentários em Strings:** texto que parece comentário dentro de strings não é tratado como tal.

```
let texto = "// Isso não é um comentário";
console.log(texto); // "// Isso não é um comentário"
```

f) Exercício Prático

Escreva um código que:

1. Use um comentário de uma linha para explicar uma variável.
2. Use um comentário de várias linhas para descrever uma função.
3. Desative uma linha de código com um comentário.

Solução:

```
// Armazena o preço base de um produto
let preco = 50;

/*
  Esta função aplica um desconto ao preço base.
  O desconto é calculado como uma porcentagem do valor original.
*/
function aplicarDesconto(valor, desconto) {
  return valor - (valor * desconto / 100);
}

// console.log(aplicarDesconto(preco, 20)); // Linha desativada para teste
console.log(aplicarDesconto(preco, 10)); // 45
```

Loops em JS

Um **loop** é uma maneira de repetir um trecho de código até que uma condição seja atendida ou um objetivo seja concluído. Os **loop** em JavaScript ajudam a evitar a reprodução manual de comandos, tornando o código mais eficiente e legível.

A **iteração**, por sua vez, refere-se a cada execução individual desse código dentro do **loop**.

Tipos de loops em JavaScript

a) Loop for

- **Sintaxe**

```
for (let i = 0; i < 5; i++) {
  console.log(i); // 0, 1, 2, 3, 4
}
```

- **Descrição:** O **loop** for é o mais comum e estruturado. Ele consiste em três partes:
 - **Inicialização:** defina uma variável de controle (ex.: let i = 0).
 - **Condição:** testada antes de cada iteração; é verdade, o loop continua (ex.: i < 5).
 - **Incremento:** atualiza uma variável de controle após cada iteração (ex.: i++).
- **Uso:** ideal quando o número de iterações é conhecido.
- **Exemplo Básico:**

```
for (let i = 0; i < 5; i++) {
  console.log(i); // 0, 1, 2, 3, 4
}
```

- **Exemplo com array:**

```
const frutas = ["maçã", "banana", "laranja"];
for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]); // maçã, banana, laranja
}
```

b) Loop while

- **Sintaxe**

```
while (condição) {
  // Código a ser repetido
}
```

- **Descrição:** executa o bloco de código enquanto a condição especificada for verdadeira. A condição é verificada antes de cada iteração.
- **Uso:** útil quando o número de iterações não é conhecido anteriormente.
- **Exemplo Básico:**

```
let contador = 0;
while (contador < 3) {
  console.log("Contador:", contador); // Contador: 0, 1, 2
  contador++;
}
```

- **Exemplo Prático:**

```
let numero = 10;
while (numero > 0) {
    console.log(numero); // 10, 9, 8, ..., 1
    numero--;
}
```

c) Loop do...while

- **Sintaxe:**

```
do {
    // Código a ser repetido
} while (condição);
```

- **Descrição:** semelhante ao while, mas a condição é verificada após cada iteração. Isso garante que o bloco de código seja executado pelo menos uma vez.
- **Uso:** ideal quando o código precisa rodar pelo menos uma vez, independentemente da condição.
- **Exemplo Básico:**

```
let x = 0;
do {
    console.log(x); // 0
    x++;
} while (x < 0); // Executa uma vez, mesmo com condição falsa
```

- **Exemplo Prático:**

```
let tentativa = 0;
do {
    console.log("Tentativa:", tentativa); // Tentativa: 0, 1, 2
    tentativa++;
} while (tentativa < 3);
```

d) Loop for...in

- **Sintaxe**

```
for (variável in objeto) {
    // Código a ser repetido
}
```

- **Descrição:** percorrer as propriedades enumeráveis de um objeto. A variável recebe o nome de cada propriedade a cada iteração.

- **Uso:** recomendado para objetos, mas pode ser usado com arrays (embora não seja a melhor prática).
- **Exemplo com Objeto:**

```
const pessoa = { nome: "Ana", idade: 25, cidade: "São Paulo" };
for (let chave in pessoa) {
    console.log(chave + ": " + pessoa[chave]);
    // nome: Ana, idade: 25, cidade: São Paulo
}
```

- **Exemplo com Array:**

```
const cores = ["vermelho", "azul"];
for (let i in cores) {
    console.log(i, cores[i]); // 0 vermelho, 1 azul
}
```

e) Loop for...of

- **Sintaxe:**

```
for (variável of iterável) {
    // Código a ser repetido
}
```

- **Descrição:** introduzido no ES6, percorre os valores de um objeto iterável (como arrays, strings ou outros iteráveis).
- **Uso:** melhor opção para arrays e estruturas iteráveis, em vez de for...in.
- **Exemplo com Array:**

```
const numeros = [10, 20, 30];
for (let valor of numeros) {
    console.log(valor); // 10, 20, 30
}
```

- **Exemplo com String:**

```
const texto = "Olá";
for (let letra of texto) {
    console.log(letra); // O, l, á
}
```

f) Controle de Loops: break e continue

- **break:** interrompe o loop completamente e sai dele.

```
for (let i = 0; i < 5; i++) {
  if (i === 3) {
    break; // Para quando i é 3
  }
  console.log(i); // 0, 1, 2
}
```

- **continue:** pula a iteração atual e vai para a próxima.

```
for (let i = 0; i < 5; i++) {
  if (i === 2) {
    continue; // Pula o 2
  }
  console.log(i); // 0, 1, 3, 4
}
```

g) Exercício Prático

Escreva um código que:

Use um loop para somar os números de 1 a 5.

Use um loop while para contar regressivamente de 3 a 1.

Use um loop for...of para exibir cada item de um array.

Solução:

```
// 1. Soma com for
let soma = 0;
for (let i = 1; i <= 5; i++) {
  soma += i;
}
console.log("Soma:", soma); // Soma: 15

// 2. Contagem regressiva com while
let contagem = 3;
while (contagem >= 1) {
  console.log(contagem); // 3, 2, 1
  contagem--;
}

// 3. for...of com array
const itens = ["caneta", "lápiz", "borracha"];
for (let item of itens) {
  console.log(item); // caneta, lápis, borracha
}
```

h) Resumo de *loops*

Type	Uso Principal	Verifica Condição	Executa ao Menos Uma Vez?
<code>for</code>	Iterações conhecidas	Antes	Não
<code>while</code>	Condições dinâmicas	Antes	Não
<code>do...while</code>	Executa pelo menos uma vez	Depois	Sim
<code>for...in</code>	Propriedades de objetos	-	-
<code>for...of</code>	Valores de iteráveis	-	-

Funções

Funções são blocos de código reutilizáveis que executam tarefas específicas ou calculam valores. Essenciais em JavaScript, elas **organizam o código, evitam repetições e aumentam a modularidade**. Podem receber parâmetros, processá-los e retornar resultados. Sendo objetos de primeira classe, podem ser armazenadas em variáveis, passadas como argumentos ou retornadas por outras funções.

Uma função é um conjunto de instruções que pode ser chamado em diferentes partes do programa. Geralmente nomeada, pode receber parâmetros e devolver um resultado com *return*, facilitando a divisão de problemas complexos em partes menores e mais gerenciáveis.

Tipos de Declaração de Funções

a) Declaração de Função (function declaration)

- **Sintaxe:**

```
function nomeDaFuncao(parametro1, parametro2) {
    // Código a ser executado
    return resultado;
}
```

- **Descrição:** define uma função com um nome explícito. É “elevada” (hoisted) ao topo do escopo, permitindo que seja chamada antes de sua definição no código.
- **Uso:** ideal para funções reutilizáveis e bem definidas.
- **Exemplo Básico:**

```
function somar(a, b) {
    return a + b;
}
console.log(somar(2, 3)); // 5
```

- **Exemplo com Hoisting:**

```
console.log(multiplicar(4, 5)); // 20
function multiplicar(x, y) {
  return x * y;
}
```

b) Expressão de Função (function expression)

- **Sintaxe:**

```
const nomeDaFuncao = function(parametro1, parametro2) {
  // Código a ser executado
  return resultado;
};
```

- **Descrição:** uma função é atribuída a uma variável. Não sofre hoisting completo – apenas a variável é elevada, mas sem o valor da função, então deve ser definida antes de ser chamada.
- **Uso:** útil para funções anônimas ou quando a função precisa ser dinâmica.
- **Exemplo Básico:**

```
const subtrair = function(a, b) {
  return a - b;
};
console.log(subtrair(10, 7)); // 3
```

- **Exemplo Anônimo:**

```
const operacao = function(x, y) {
  return x * y;
};
console.log(operacao(3, 4)); // 12
```

c) Funções de Seta (arrow functions)

- **Sintaxe:**

```
const nomeDaFuncao = (parametro1, parametro2) => {
  // Código a ser executado
  return resultado;
};
```

- **Descrição:** introduzidas no ES6, oferecem uma *sintaxe* mais concisa e um comportamento especial com o *this* (não possuem seu próprio *this*, herdam do escopo externo).
- **Uso:** perfeitas para funções curtas ou em *callbacks*.

- **Exemplo Básico:**

```
const dividir = (a, b) => {
  return a / b;
};
console.log(dividir(8, 2)); // 4
```

- **Exemplo Conciso (sem chaves):**

```
const dobrar = x => x * 2; // Retorno implícito
console.log(dobrar(5)); // 10
```

d) Parâmetros e Retorno

- **Parâmetros:** variáveis que a função recebe como entrada. Podem ter valores padrão (ES6).

```
function saudar(nome = "Visitante") {
  return `Olá, ${nome}!`;
}
console.log(saudar("Ana")); // "Olá, Ana!"
console.log(saudar()); // "Olá, Visitante!"
```

- **Retorno:** o valor que a função devolve com *return*. Sem *return*, a função retorna *undefined* por padrão.

```
function semRetorno(a, b) {
  a + b; // Não retorna nada
}
console.log(semRetorno(1, 2)); // undefined
```

- **Parâmetros Rest (...):** permite receber um número variável de argumentos como um array.

```
function somarTudo(...numeros) {
  return numeros.reduce((total, num) => total + num, 0);
}
console.log(somarTudo(1, 2, 3, 4)); // 10
```

e) Escopo e Closure

- **Escopo:** funções criam seu próprio escopo. Variáveis declaradas dentro delas não são acessíveis fora.

```
function exemploEscopo() {
    let local = "Eu sou local";
    console.log(local); // "Eu sou local"
}
exemploEscopo();
// console.log(local); // ReferenceError
```

- **Closure:** funções internas podem “lembrar” variáveis do escopo externo mesmo após a função externa terminar.

```
function criarContador() {
    let contador = 0;
    return function() {
        return contador++;
    };
}
const meuContador = criarContador();
console.log(meuContador()); // 0
console.log(meuContador()); // 1
```

f) Exercício Prático

Escreva um código que:

1. Crie uma função declarada para calcular o perímetro de um retângulo.
2. Use uma expressão de função para verificar se um número é par.
3. Use uma arrow function para concatenar duas strings.

Solução:

```
// 1. Declaração de função
function calcularPerimetro(largura, altura) {
    return 2 * (largura + altura);
}
console.log("Perímetro:", calcularPerimetro(5, 3)); // Perímetro: 16

// 2. Expressão de função
const ehPar = function(numero) {
    return numero % 2 === 0;
};
console.log(ehPar(4)); // true
console.log(ehPar(7)); // false

// 3. Arrow function
const juntarTextos = (texto1, texto2) => texto1 + " " + texto2;
console.log(juntarTextos("Olá", "Mundo")); // "Olá Mundo"
```

g) Resumo das Funções

Tipo	Característica Principal	Hoisting?	this Próprio?
function declaration	Nomeada, elevada ao escopo	Sim	Sim
function expression	Atribuída a variável, anônima	Não	Sim
arrow function	Sintaxe curta, herda this	Não	Não

Objetos JSON

JSON (JavaScript Object Notation) é um formato leve e amplamente utilizado para armazenar e trocar dados entre sistemas. Apesar de ter origem no JavaScript, ele é independente de linguagem, sendo suportado por diversas tecnologias.

Em JavaScript, objetos JSON são frequentemente **usados para representar dados estruturados** de forma simples e legível, tanto para humanos quanto para máquinas.

Vamos então explorar o conceito de JSON e os tipos de dados que ele suporta.

JSON é um formato de texto baseado em pares chave-valor, inspirado na sintaxe de objetos JavaScript, mas com regras mais rígidas. Ele é usado para serializar dados (convertê-los em texto) e desserializá-los (convertê-los de volta em objetos), facilitando a comunicação entre cliente e servidor, armazenamento de configurações ou troca de informações.

- **Características Principais:**
 - Estrutura baseada em pares chave: valor.
 - As chaves são sempre strings entre aspas duplas (").
 - Os valores podem ser de tipos específicos (veja abaixo).
 - Não suporta funções, métodos ou comentários – apenas dados puros.
- **Sintaxe Básica:**

```
{
  "chave": "valor",
  "outraChave": 123
}
```

- **Uso em JavaScript:**
 - Para trabalhar com JSON, usamos os métodos JSON.stringify() (converte um objeto JS em string JSON) e JSON.parse() (converte uma string JSON em objeto JS).

Exemplo Básico:

```
// Objeto JavaScript
const pessoa = {
  nome: "Maria",
  idade: 30,
  cidade: "Rio de Janeiro"
};

// Convertendo para JSON (string)
const jsonString = JSON.stringify(pessoa);
console.log(jsonString);
// {"nome": "Maria", "idade": 30, "cidade": "Rio de Janeiro"}

// Convertendo de volta para objeto
const objeto = JSON.parse(jsonString);
console.log(objeto.nome); // "Maria"
```

• Casos de Uso:

- APIs REST: Dados enviados e recebidos em formato JSON.
- Configurações: Armazenar informações em arquivos.json.
- Comunicação: Transferência de dados entre sistemas.

Tipos de dado em JSON

JSON suporta um conjunto limitado de tipos de dados, garantindo simplicidade e compatibilidade entre linguagens. Abaixo estão os tipos permitidos, com exemplos:

a) String

- **Descrição:** sequências de caracteres entre aspas duplas (""). Suporta caracteres Unicode.
- **Exemplo:**

```
{
  "nome": "João",
  "mensagem": "Olá, mundo!"
}
```

b) Number

- **Descrição:** números inteiros ou decimais, sem distinção entre int e float. Não suporta NaN ou Infinity.
- **Exemplo:**

```
{
  "idade": 25,
  "altura": 1.75
}
```

c) Boolean

- **Descrição:** valores lógicos true ou false, escritos sem aspas.
- **Exemplo:**

```
{
  "ativo": true,
  "verificado": false
}
```

d) Null

- **Descrição:** representa a ausência de valor, escrito como null (sem aspas).
- **Exemplo:**

```
{
  "endereco": null,
  "telefone": "1234-5678"
}
```

e) Object

- **Descrição:** coleção de pares chave-valor, delimitada por chaves {}. Pode conter outros objetos (aninhamento).
- **Exemplo:**

```
{
  "pessoa": {
    "nome": "Ana",
    "idade": 28
  },
  "contato": {
    "email": "ana@example.com"
  }
}
```

f) Array

- **Descrição:** lista ordenada de valores, delimitada por colchetes []. Pode conter qualquer tipo de dado JSON.
- **Exemplo:**

```
{
  "hobbies": ["leitura", "esportes", "viagens"],
  "notas": [10, 8.5, 9]
}
```

- **Restrições:**

- Funções, datas (Date), expressões regulares (RegExp) ou valores como undefined não são suportados em JSON. Esses tipos são específicos do JavaScript e perdidos ao converter para JSON.

- **Exemplo Completo:**

```
const dados = {
  nome: "Carlos",
  idade: 35,
  casado: false,
  filhos: null,
  enderecos: [
    { rua: "Avenida Principal", numero: 100 },
    { rua: "Rua Secundária", numero: 50 }
  ],
  notas: [8, 9.5, 7]
};

// Convertendo para JSON
const jsonDados = JSON.stringify(dados);
console.log(jsonDados);
// {"nome":"Carlos","idade":35,"casado":false,"filhos":null,"enderecos":[{"rua":"Avenida

// Convertendo de volta
const objetoDados = JSON.parse(jsonDados);
console.log(objetoDados.enderecos[0].rua); // "Avenida Principal"
```

g) Exercício Prático

Crie um objeto JSON representando um produto com os seguintes dados:

- Nome (string), preço (number), em estoque (boolean), categorias (array), detalhes (objeto com marca e peso). Converta-o para string e depois de volta para objeto.

Solução:

```
const produto = {
  nome: "Celular",
  preco: 1200.99,
  emEstoque: true,
  categorias: ["eletrônicos", "telefonia"],
  detalhes: {
    marca: "TechX",
    peso: 0.15
  }
};

// Para JSON
const jsonProduto = JSON.stringify(produto);
console.log(jsonProduto);

// De volta para objeto
const objProduto = JSON.parse(jsonProduto);
console.log(objProduto.detalhes.marca); // "TechX"
```

Manipulação de Arrays com Map, Filter e Reduce

Os métodos map, filter e reduce são ferramentas poderosas em JavaScript para manipular arrays de forma funcional, ou seja, sem alterar o array original e com uma abordagem mais declarativa. Eles permitem transformar, filtrar e agregar dados de maneira eficiente e legível, sendo amplamente utilizados em programação moderna. Neste tópico, exploraremos cada um desses métodos com base em exemplos práticos inspirados no artigo da Alura, usando um cenário de empresas como contexto.

Esses métodos fazem parte do paradigma funcional e operam sobre arrays, retornando novos arrays ou valores sem modificar os dados originais. Cada um tem um propósito específico:

- **map:** transforma cada elemento do array e retorna um novo array com os resultados.
- **filter:** filtra elementos com base em uma condição, retornando um novo array apenas com os itens que a satisfazem.
- **reduce:** reduz o array a um único valor, aplicando uma operação acumulativa.

Eles são especialmente úteis para evitar *loops* manuais (*for*, *while*) e tornar o código mais conciso e expressivo.

Métodos de Manipulação de Arrays

a) Método map

- **Sintaxe:**

```
array.map((elemento, índice, array) => {
    // Transformação do elemento
    return novoValor;
});
```

- **Descrição:** aplica uma função a cada elemento do array e retorna um novo array com os resultados. O array original permanece intacto.
- **Uso:** Ideal para criar uma nova lista com dados transformados.
- **Exemplo do Artigo da Alura:** imagine que temos um array de CEOs de empresas e queremos criar uma lista com o nome de cada empresa e seu respectivo CEO:

```
const empresas = [
  { nome: "Samsung", valorDeMercado: 50, CEO: "Kim Hyun Suk", anoDeCriacao: 1938 },
  { nome: "Microsoft", valorDeMercado: 415, CEO: "Satya Nadella", anoDeCriacao: 1975 },
  { nome: "Intel", valorDeMercado: 117, CEO: "Brian Krzanich", anoDeCriacao: 1968 },
  { nome: "Facebook", valorDeMercado: 383, CEO: "Mark Zuckerberg", anoDeCriacao: 2004 },
  { nome: "Spotify", valorDeMercado: 30, CEO: "Daniel Ek", anoDeCriacao: 2006 },
  { nome: "Apple", valorDeMercado: 845, CEO: "Tim Cook", anoDeCriacao: 1976 }
];

const listaCEOs = empresas.map(empresa => `${empresa.nome} CEO: ${empresa.CEO}`);
console.log(listaCEOs);
// [
//   "Samsung CEO: Kim Hyun Suk",
//   "Microsoft CEO: Satya Nadella",
//   "Intel CEO: Brian Krzanich",
//   "Facebook CEO: Mark Zuckerberg",
//   "Spotify CEO: Daniel Ek",
//   "Apple CEO: Tim Cook"
// ]
```

- **Detalhe:** o map mantém o mesmo número de elementos, apenas transformando cada um conforme a função fornecida.

b) Método filter

- **Sintaxe:**

```
array.filter((elemento, índice, array) => {
    // Condição booleana
    return verdadeiroOuFalso;
});
```

- **Descrição:** cria um novo array contendo apenas os elementos que passam no teste da função (retornam true).
- **Uso:** Perfeito para selecionar subconjuntos de dados baseados em critérios.

- **Exemplo do Artigo da Alura:** queremos listar apenas as empresas criadas após o ano 2000:

```
const empresasNovas = empresas.filter(empresa => empresa.anoDeCriacao > 2000);
console.log(empresasNovas);
// [
//   { nome: "Facebook", valorDeMercado: 383, CEO: "Mark Zuckerberg", anoDeCriacao: 2004
//   { nome: "Spotify", valorDeMercado: 30, CEO: "Daniel Ek", anoDeCriacao: 2006 }
// ]
```

- **Detalhe:** o filter reduz o tamanho do array, retornando apenas os elementos que atendem à condição.

c) Método reduce

- **Sintaxe:**

```
array.reduce((acumulador, elemento, índice, array) => {
  // Operação de redução
  return novoAcumulador;
}, valorInicial);
```

- **Descrição:** reduz o array a um único valor, aplicando uma função que acumula resultados a cada iteração. O valorInicial é opcional, mas essencial em alguns casos.
- **Uso:** útil para somar valores, concatenar dados ou criar um resultado único.
- **Exemplo do Artigo da Alura:** vamos somar o valor de mercado de todas as empresas:

```
const valorTotal = empresas.reduce((acumulado, empresa) => {
  return acumulado + empresa.valorDeMercado;
}, 0);
console.log(valorTotal); // 1840
```

- **Detalhe:** o reduce percorre o array, somando o valorDeMercado ao acumulado, começando de 0 (valor inicial). Sem o valor inicial, o primeiro elemento seria usado como base.

d) Combinando map, filter e reduce

Esses métodos podem ser encadeados para resolver problemas mais complexos. Por exemplo, podemos filtrar empresas criadas após 2000 e somar seus valores de mercado:

```
const totalEmpresasNovas = empresas
  .filter(empresa => empresa.anoDeCriacao > 2000)
  .reduce((acumulado, empresa) => acumulado + empresa.valorDeMercado, 0);
console.log(totalEmpresasNovas); // 413 (383 do Facebook + 30 do Spotify)
```

Ou usar map após filter para transformar os dados filtrados:

```
const nomesEmpresasNovas = empresas
  .filter(empresa => empresa.anoDeCriacao > 2000)
  .map(empresa => empresa.nome);
console.log(nomesEmpresasNovas); // ["Facebook", "Spotify"]
```

e) Exercício Prático

Com base no array empresas, escreva um código que:

1. Use filter para selecionar empresas com valor de mercado acima de 100.
2. Use map para criar uma lista com os nomes e CEOs dessas empresas.
3. Use reduce para somar o valor de mercado dessas empresas.

Solução:

```
const empresasValiosas = empresas.filter(empresa => empresa.valorDeMercado > 100);
const listaCEOsValiosos = empresasValiosas.map(empresa => `${empresa.nome} CEO: ${empres
const totalValiosas = empresasValiosas.reduce((acumulado, empresa) => acumulado + empres

console.log(listaCEOsValiosos);
// [
//   "Microsoft CEO: Satya Nadella",
//   "Intel CEO: Brian Krzanich",
//   "Facebook CEO: Mark Zuckerberg",
//   "Apple CEO: Tim Cook"
// ]
console.log(totalValiosas); // 1760
```

f) Resumo dos Métodos

Método	Uso Principal	Retorno	Modifica o Original?
map	Transformar elementos	Novo array	Não
filter	Filtrar elementos	Novo array filtrado	Não
reduce	Reduzir a um único valor	Valor único	Não

JSON.stringify e JSON.parse em JavaScript

JSON (JavaScript Object Notation) é um formato amplamente utilizado para representar e trocar dados. Em JavaScript, os métodos JSON.stringify e JSON.parse são ferramentas fundamentais para converter objetos JavaScript em strings JSON e vice-versa. Esses métodos permitem serializar dados para armazenamento ou transmissão e desserializá-los para uso no código.

Neste tópico, exploraremos como eles funcionam, suas opções e exemplos práticos.

- **JSON.stringify**: converte um valor JavaScript (como um objeto, array ou tipo primitivo) em uma string no formato JSON. Esse processo é chamado de **serialização**, pois transforma dados complexos em texto que pode ser facilmente armazenado ou enviado.
- **JSON.parse**: converte uma string JSON válida de volta em um valor JavaScript (objeto, array, etc.). Esse processo é chamado de **desserialização**, restaurando os dados para uso no programa.

Esses métodos são essenciais em cenários como comunicação com APIs, armazenamento local (ex.: localStorage) ou troca de dados entre sistemas.

Funcionamento e Uso

a) JSON.stringify

- **Sintaxe:**

```
JSON.stringify(valor, replacer?, space?)
```

- valor: o valor JavaScript a ser convertido (objeto, array, string, número, etc.).
- replacer (opcional): uma função ou array que filtra ou transforma os valores durante a serialização.
- space (opcional): um número ou string que define a indentação da string resultante para melhor legibilidade.
- **Descrição**: transforma o valor em uma string JSON. Tipos não suportados (como funções ou undefined) são omitidos ou substituídos por null.
- **Uso**: preparar dados para envio ou armazenamento.
- **Exemplo Básico**:

```
const pessoa = {
  nome: "João",
  idade: 25,
  ativo: true
};
const jsonString = JSON.stringify(pessoa);
console.log(jsonString);
// {"nome":"João","idade":25,"ativo":true}
```

- **Exemplo com Indentação:**

```
const jsonFormatado = JSON.stringify(pessoa, null, 2);
console.log(jsonFormatado);
// {
//   "nome": "João",
//   "idade": 25,
//   "ativo": true
// }
```

- **Exemplo com Funções e undefined:**

```
const obj = {
  nome: "Ana",
  metodo: function() { console.log("Oi"); },
  indefinido: undefined
};
console.log(JSON.stringify(obj));
// {"nome": "Ana"} (função e undefined são ignorados)
```

- **Exemplo com replacer:**

```
const dados = { nome: "Carlos", senha: "12345" };
const jsonSemSenha = JSON.stringify(dados, (chave, valor) => {
  if (chave === "senha") return undefined; // Remove a chave "senha"
  return valor;
});
console.log(jsonSemSenha);
// {"nome": "Carlos"}
```

b) JSON.parse

- **Sintaxe:**

JSON.parse(texto, reviver?)

- texto: a string JSON a ser convertida.
- reviver (opcional): uma função que transforma os valores durante a desserialização.
- **Descrição:** converte uma string JSON em um valor JavaScript. A string deve seguir a sintaxe JSON válida, ou um erro (SyntaxError) será lançado.
- **Uso:** restaurar dados recebidos ou armazenados.
- **Exemplo Básico:**

```
const json = '{"nome":"Maria","idade":30}';
const objeto = JSON.parse(json);
console.log(objeto.nome); // "Maria"
console.log(objeto.idade); // 30
```

- **Exemplo com reviver:**

```
const jsonData = '{"data":"2023-10-15","valor":100}';
const objData = JSON.parse(jsonData, (chave, valor) => {
  if (chave === "data") return new Date(valor); // Converte string em objeto Date
  return valor;
});
console.log(objData.data); // Objeto Date: 2023-10-15T00:00:00.000Z
```

- **Exemplo com Erro:**

```
try {
  const jsonInvalido = "{nome: 'Pedro'}"; // Sintaxe inválida (sem aspas nas chaves)
  JSON.parse(jsonInvalido);
} catch (e) {
  console.log("Erro:", e); // SyntaxError: Unexpected token n in JSON
}
```

c) Exercício Prático

Crie um código que:

1. Defina um objeto com nome, idade e hobbies (array).
2. Converta-o para uma string JSON com indentação.
3. Converta a string de volta para um objeto e acesse um hobby específico.

Solução:

```
// 1. Definindo o objeto
const usuario = {
  nome: "Lucas",
  idade: 28,
  hobbies: ["futebol", "leitura", "viagens"]
};

// 2. Convertendo para JSON com indentação
const jsonUsuario = JSON.stringify(usuario, null, 2);
console.log(jsonUsuario);
// {
//   "nome": "Lucas",
//   "idade": 28,
//   "hobbies": [
//     "futebol",
//     "leitura",
//     "viagens"
//   ]
// }

// 3. Convertendo de volta e acessando um hobby
const objUsuario = JSON.parse(jsonUsuario);
console.log(objUsuario.hobbies[1]); // "leitura"
```

d) Resumo dos Métodos

Método	Função Principal	Parâmetros Opcionais	Retorno
<code>JSON.stringify</code>	Serializa valor em string JSON	<code>replacer</code> , <code>space</code>	String
<code>JSON.parse</code>	Desserializa string em valor JS	<code>reviver</code>	Objeto/valor JS

OPERAÇÕES ASSÍNCRONAS EM JAVASCRIPT

JavaScript é uma linguagem de programação single-threaded, ou seja, **executa uma tarefa por vez em uma única thread**. No entanto, muitas operações – como requisições a APIs, leitura de arquivos ou temporizações – podem levar tempo para serem concluídas. Se essas tarefas fossem executadas de forma síncrona (bloqueando a execução até terminarem), a experiência do usuário seria prejudicada, com interfaces travadas ou tempos de espera desnecessários. Para resolver isso, **JavaScript utiliza operações assíncronas**, permitindo que o código continue sendo executado enquanto espera por resultados de tarefas demoradas.

Em termos simples, uma thread é como um cozinheiro que só pode fazer uma coisa de cada vez. **JavaScript ser *single-threaded* significa que ele executa uma tarefa por vez, o que pode causar problemas se uma tarefa demorar muito. Para resolver isso, JavaScript usa truques (como ter ajudantes que fazem o trabalho pesado em segundo plano) para garantir que a interface do usuário não trave.**

Operações assíncronas são baseadas no **event loop**, um mecanismo que gerencia a execução de código assíncrono em JavaScript. Elas possibilitam que o programa delegue tarefas (como buscar dados de um servidor) e continue processando outras instruções, retornando ao resultado apenas quando ele estiver pronto. Esse modelo é essencial para aplicações web modernas, onde a responsividade é crucial.

Inicialmente, operações assíncronas em JavaScript eram gerenciadas com **callbacks**, funções passadas como argumentos para serem executadas quando uma tarefa terminasse. Contudo, o uso excessivo de *callbacks* podia levar ao chamado "*callback hell*", tornando o código difícil de ler e manter. Com o tempo, evoluções como **Promises** (introduzidas no ES6) e a sintaxe **async/await** (ES8) trouxeram soluções mais elegantes e legíveis, permitindo lidar com assincronismo de forma semelhante a código síncrono, mas sem bloqueios.

Exemplo Simples de Assincronismo:

```
console.log("Início");
setTimeout(() => {
  console.log("Tarefa assíncrona concluída");
}, 2000); // Executa após 2 segundos
console.log("Fim");
// Saída: "Início" -> "Fim" -> (2s depois) "Tarefa assíncrona concluída"
```

a) Promises

Uma **Promise** é um objeto que representa a eventual conclusão (ou falha) de uma operação assíncrona e seu valor resultante.

Ela é uma promessa de que algo será resolvido no futuro, seja com sucesso (resolvida) ou com erro (rejeitada). *Promises* foram introduzidas no ES6 para substituir *callbacks* aninhados, oferecendo uma abordagem mais estruturada e encadeável.

• Estados:

- **Pending (pendente)**: estado inicial, ainda não resolvida nem rejeitada.
- **Fulfilled (resolvida)**: a operação foi concluída com sucesso.
- **Rejected (rejeitada)**: a operação falhou.

- **Sintaxe:**

```
const minhaPromise = new Promise((resolve, reject) => {
  // Código assíncrono
  if (/* sucesso */) {
    resolve(valor); // Resolve a Promise
  } else {
    reject(erro); // Rejeita a Promise
  }
});
```

- **Métodos:**

- .then(): executa uma função quando a Promise é resolvida.
- .catch(): trata erros quando a Promise é rejeitada.
- .finally(): executa uma função independentemente do resultado.

- **Exemplo Básico:**

```
const promessa = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Dados recebidos!");
  }, 1000);
});
```

```
promessa
  .then(resultado => console.log(resultado)) // "Dados recebidos!" após 1s
  .catch(erro => console.log("Erro:", erro));
```

- **Exemplo com erro:**

```
const promessaErro = new Promise((resolve, reject) => {
  setTimeout(() => {
    reject("Falha na conexão");
  }, 1000);
});
```

```
promessaErro
  .then(resultado => console.log(resultado))
  .catch(erro => console.log("Erro:", erro)); // "Erro: Falha na conexão" após 1s
```

- **Encadeamento:**

```
const buscarUsuario = new Promise((resolve) => {
  setTimeout(() => resolve({ id: 1, nome: "Ana" }), 1000);
});

buscarUsuario
  .then(usuario => {
    console.log("Usuário:", usuario.nome); // "Usuário: Ana"
    return usuario.id;
  })
  .then(id => console.log("ID:", id)) // "ID: 1"
  .catch(erro => console.log(erro));
```

- **Uso prático:** Promises são comuns em APIs como fetch para requisições HTTP:

```
fetch("https://api.exemplo.com/dados")
  .then(resposta => resposta.json())
  .then(dados => console.log(dados))
  .catch(erro => console.log("Erro:", erro));
```

b) Async e Await

A sintaxe **async/await**, introduzida no ES8, **é uma camada sobre Promises que torna o código assíncrono mais legível e semelhante a código síncrono.**

Uma função marcada com **async** automaticamente retorna uma Promise, e dentro dela, o operador **await** pausa a execução até que uma Promise seja resolvida ou rejeitada.

- **Sintaxe:**

```
async function nomeDaFuncao() {
  const resultado = await promessa;
  return resultado;
}
```

- **Características:**

- **async:** declara uma função como assíncrona.
- **await:** só pode ser usado dentro de funções **async** e espera a resolução de uma Promise.

- **Exemplo Básico:**

```
async function obterDados() {
    const promessa = new Promise(resolve => {
        setTimeout(() => resolve("Dados obtidos!"), 1000);
    });
    const resultado = await promessa;
    console.log(resultado); // "Dados obtidos!" após 1s
}
obterDados();
```

- **Exemplo com tratamento de erro:**

```
async function verificarConexao() {
    try {
        const promessa = new Promise((resolve, reject) => {
            setTimeout(() => reject("Sem conexão"), 1000);
        });
        const resultado = await promessa;
        console.log(resultado);
    } catch (erro) {
        console.log("Erro:", erro); // "Erro: Sem conexão" após 1s
    }
}
verificarConexao();
```

- **Exemplo com fetch¹:**

```
async function buscarDadosAPI() {
    try {
        const resposta = await fetch("https://api.exemplo.com/dados");
        const dados = await resposta.json();
        console.log(dados);
    } catch (erro) {
        console.log("Erro na requisição:", erro);
    }
}
buscarDadosAPI();
```

- **Vantagem:** evita o encadeamento excessivo de .then(), tornando o código mais claro e fácil de seguir.

c) Exercício Prático

Crie um código que:

¹ A API do Fetch será abordada no capítulo Javascript no lado cliente (Web Browsers).

1. Use uma Promise para simular o carregamento de um usuário após 2 segundos.
2. Use async/await para exibir o nome do usuário ou tratar um erro.

Solução:

```
const carregarUsuario = new Promise((resolve, reject) => {
  setTimeout(() => {
    const sucesso = true; // Altere para false para testar erro
    if (sucesso) {
      resolve({ id: 1, nome: "Pedro" });
    } else {
      reject("Usuário não encontrado");
    }
  }, 2000);
});

async function exibirUsuario() {
  try {
    const usuario = await carregarUsuario;
    console.log("Nome:", usuario.nome); // "Nome: Pedro" após 2s
  } catch (erro) {
    console.log("Erro:", erro);
  }
}

exibirUsuario();
```

d) Resumo das operações assíncronas

Técnica	Uso Principal	Sintaxe Principal	Tratamento de Erros
Promise	Gerenciar assincronismo	.then(), .catch()	.catch()
async/await	Simplificar código assíncrono	await em função async	try...catch

JAVASCRIPT NO LADO CLIENTE (WEB BROWSERS)

FETCH API

Antes de começarmos a falar sobre Fetch API é importante explicarmos brevemente o que é uma requisição AJAX.

AJAX (Asynchronous JavaScript and XML) é uma técnica usada para criar aplicações web mais dinâmicas e interativas, permitindo a comunicação com o servidor sem a

necessidade de recarregar a página inteira. Com AJAX, os sites podem atualizar partes específicas do conteúdo de forma assíncrona, melhorando a experiência do usuário.

Assim, quando você entra em uma página, consegue perceber ela sendo carregada, inclusive o seu navegador mostra que ela está sendo carregada. Você inclusive percebe se clicar no botão de voltar do navegador. Ele irá voltar até a página antiga. Quando uma requisição AJAX ocorre, não há nenhuma troca de página, apenas o carregamento de informações é realizado “por trás das câmeras”. Normalmente depois desses dados chegarem, um código JS altera dinamicamente a página para mostrar os novos dados. MAGICA!!!

E o que o Fetch tem a ver com isso? Essa API é um tipo de AJAX.

A **Fetch API** é uma **forma moderna e simplificada de fazer requisições HTTP em JavaScript**. Ela permite buscar dados de servidores de forma assíncrona, sem precisar recarregar a página. Utiliza a função `fetch()`, que retorna uma Promise, facilitando o tratamento de respostas e erros de maneira mais limpa e legível do que o antigo XMLHttpRequest (veja no próximo tópico).

Por exemplo, para buscar dados de uma API, basta usar:

```
fetch('https://swapi.dev/api')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Erro:', error));
```

Para saber mais sobre fetch acesse: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

XML HTTP REQUEST

O **XMLHttpRequest (XHR)** é um objeto JavaScript que permite a comunicação assíncrona com servidores web. Ele possibilita o envio e recebimento de dados sem a necessidade de recarregar a página, sendo a base para requisições AJAX.

Principais métodos e propriedades:

- `.open(method, url, async)`: configura a requisição (GET, POST, etc.).
- `.send(body)`: envia a requisição ao servidor.
- `.onreadystatechange`: define uma função para monitorar mudanças no estado da requisição.
- `.responseText`: contém a resposta como texto.
- `.status`: retorna o código HTTP da resposta (200, 404, etc.).

Apesar de ainda ser possível de ser utilizado, o XMLHttpRequest foi colocado em desuso com a entrada do fetchApi

Exemplo

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "https://swapi.dev/api/people/1", true);
xhr.onreadystatechange = function () {
    if (xhr.readyState === 4 && xhr.status === 200) {
        console.log(JSON.parse(xhr.responseText)); // Converte resposta para JSON
    }
};
xhr.send();
```

Para saber mais sobre o XML Http Request acesse:

<https://developer.mozilla.org/pt-BR/docs/Web/API/XMLHttpRequest>

HTML BÁSICO

CONCEITO

- **HTML (*HyperText Markup Language*)** é a linguagem padrão para criar páginas web.
- Estrutura o conteúdo da web usando elementos e **tags**.

Estrutura de um arquivo HTML

```
<!DOCTYPE html>
<html>
<head>
<title>Título da Página</title>
</head>
<body>
<!-- Conteúdo da página -->
</body>
</html>
```

<!DOCTYPE html>: declara o tipo de documento.

<html>: elemento raiz do documento.

<head>: contém metadados, como <title>, <meta>, <link>.

<body>: contém o conteúdo visível da página.

Estrutura Básica dos Arquivos HTML

Os arquivos recebem a extensão.html. A primeira página a ser exibida geralmente recebe o nome de **index.htm** ou **html**, dependendo do provedor que hospeda a página. Basicamente, os arquivos html seguem a seguinte estrutura:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
<html>
```

```
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1">
<title>Untitled Document</title>
</head>
<body>
<!-- Corpo da página -->
</body>
</html>
```

A declaração **<!DOCTYPE>** não é um elemento da linguagem. A sua finalidade é informar ao *Web Browser* qual a versão exata da linguagem em que a página está escrita. Quando utilizada, esta declaração deve ser a primeira coisa que aparece no documento.

TAGS HTML

1. Tags Iniciais

<HTML> ... </HTML>

Definição do início e fim do código html.

<HEAD>... </HEAD>

A tag <HEAD> indica que você declarou um cabeçalho. Atributos: Nenhum.

<BODY>... </BODY>

Definição do corpo do documento HTML. Tag obrigatório para páginas que não usem Frames. Atributos:

<DIV>... </DIV>

Indica divisões em um documento e pode ser usado para agrupar elementos em um bloco. Atributo:

- ALIGN = LEFT | RIGHT | CENTER | JUSTIFY

Especifica alinhamento horizontal padrão para o conteúdo incluído.

Obs.: Use <CENTER> em vez de <DIV ALIGN=CENTER>.

Obs.: <DIV> não pode ser usado com <P> porque um elemento <DIV> terminará em um parágrafo.

<H1>... </H1> – <H6>... </H6>

Títulos e subtítulos. Ajudam a adicionar estrutura e divisões ao seu documento tornando-o organizado.

<TITLE>... </TITLE>

Inclui um título de um documento HTML, que aparecerá na barra de título da janela do browser.

<!--... -->

Inclui um comentário no código fonte de um documento HTML.

<META>

O elemento <meta> fornece metainformação, ou seja: dá informação que descreve a informação que está contida no corpo do documento.

2. Tags para Formatação

**
** Quebra de linha forçada no texto. Para textos que devem ser quebrados em locais específicos

<CITE>... </CITE> Para destacar citações.

<CODE>... </CODE> Exemplos de código de programa.

**... ** Negrito

**... ** Fortemente enfatizado

<I>... </I> Itálico

<STRIKE>... </STRIKE> Texto riscado

<TT>... </TT> Caracteres semelhantes aos de uma máquina de escrever

<U>... </U> Texto sublinhado

<S>... </S> Texto tachado

<VAR>... </VAR> Destaca nomes de variáveis ou argumentos nos comandos.

<BIG>... </BIG> Grande

<SMALL>... </SMALL> Pequeno

_{...} Subscrito

^{...} Sobrescrito

<P> (</P> opcional) Parágrafos. Quebra de linha com espaço de uma linha em branco. Atributo:

- ALIGN = LEFT | RIGHT | CENTER | JUSTIFY

Especifica alinhamento horizontal padrão para o conteúdo incluído.

**... ** Define o tamanho, a fonte e a cor do texto inserido. Atributos:

- COLOR = #RRGGBB | NOME_DA_COR

Define a cor da fonte.

- FACE = "nomefonte1, nomefonte2, nomefonte3"

Define a tipologia, ou seja, com que tipo de fonte será mostrado o texto no browser.

- SIZE = "número"

Tamanho da fonte (entre 1 e 7). Um sinal de adição ou subtração antes do número significa tamanho relativo à configuração de fonte atual.

EXEMPLO

<CENTER>... </CENTER> Indica que o texto deve ser centralizado horizontalmente.

Atributos: Nenhum.

3. Tags de Links

- **<A>...** define um vínculo clicável para outro recurso Web ou para um ponto específico em uma página Web.
- Para construir um *link* que aponta para um endereço da Web você usa uma tag com o atributo href.

```
<a href="http://www.ufjf.br/">Aqui um link para o site da UFJF</a>
```

- Dessa forma aparecerá no navegador assim:

Aqui um link para o site da UFJF

A seguir tem-se um exemplo de link para o site do Google.

```
1 <html>
2   <body>
3     <a href="http://www.google.com.br">
4       Procura pela informação desejada no site do Google
5     </a>
6   </body>
7 </html>
```

- O que mais poderei referenciar?

e) mail por exemplo

```
<a href="mailto:pquintao@gmail.com">E-mail para Patrícia</a>
```

que será mostrado no navegador como um link qualquer:

E-mail para Patrícia

Exemplo de um link para E-Mail

```
1 <html>
2   <body>
3     <a href="mailto:coordccc@ice.ufjf.br">
4       Envie um e-mail para a Coordenação.
5     </a>
6   </body>
7 </html>
```


- Um atributo muito útil é o **title**, que você poderá usar para que apareça uma *explicação sobre o link quando você passar o mouse sobre o mesmo*:

```
<a href=mailto:pquintao@gmail.com title = "Envie um e-mail para pquintao@gmail.com">E-mail para Patrícia</a>
```

que será mostrado no navegador como um link qualquer:

E-mail para Patrícia

- Para criar um link você não precisa necessariamente apontar para um endereço na web, podemos criar **links para nossa página** da mesma forma se o arquivo teste2.html estiver no mesmo diretório da página escrita:

```
<a href="teste2.html">Aqui um link para a pagina 2</a>
```

- Este *link* aparecerá no navegador assim:

Aqui um link para a pagina 2

- Se o arquivo que se deseja "linkar" estiver em algum subdiretório devemos fazer assim:

```
<a href="subdiretorio/teste2.html">Aqui um link para a pagina 2</a>
```

que aparecerá da mesma forma no navegador:

Aqui um link para a pagina 2

4. Tags de Lista

- Uma coleção de elementos do mesmo tipo.
- Na linguagem HTML existem elementos específicos para a criação de listas, que podem ser:
 - **listas ordenadas** (, funcionam como numeração,
 - **listas não ordenadas** () e
 - **listas de definição** (<DL>).

Listas Ordenadas

- A estrutura de uma lista ordenada é bastante simples: entre os elementos de **início** `<ol>` e de **fim** `</ol>`, os itens da lista são precedidos por elementos `<li>`.
- Os itens são apresentados em linhas consecutivas e precedidos por uma numeração atribuída.

Exemplo de Listas Ordenadas

- ▶ Tags `<ol></ol>` e `<li></li>` para listas ordenadas:

```
<ol>
  <li>Um item de lista </li>
  <li>Outro item de lista </li>
</ol>
```

- ▶ que aparecerá assim no seu navegador:

```
1. Um item de lista
2. Outro item de lista
```

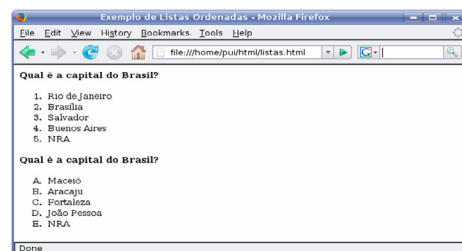
Listas Ordenadas

- ▶ **Sintaxe:**
`<ol type=... start=...> <li>item1 <li>item2 ... </ol>`
- ▶ O atributo opcional **TYPE** define como será o tipo de numeração de cada linha.
- ▶ Os tipos disponíveis são:
 - ▶ "A" (A, B, ..., Z),
 - ▶ "a" (a, b, ..., z),
 - ▶ "I" (I, II, III, IV, V, ...),
 - ▶ "i" (i, ii, iii, iv, v, ...) e
 - ▶ "1" (1, 2, 3, ...).
- ▶ Se omitido, é utilizado o padrão 1, 2, 3, ...

Listas Ordenadas

- ▶ **Sintaxe:**
`<ol type=... start=...> <li>item1 <li>item2 ... </ol>`
- ▶ O atributo opcional **START** define a partir de que elemento a numeração deve se iniciar.
- ▶ Ela deve receber como valor um número indicando em que posição a contagem deve se iniciar. A partir daí a sequência é gerada automaticamente pelo interpretador HTML.

Exemplos de Listas Ordenadas



```

1 <html>
2   <head>
3     <title> Exemplo de Listas Ordenadas </title>
4   </head>
5
6   <body>
7     <font size="2" face="Verdana">
8       <b>Qual é a capital do Brasil?</b>
9     <ol>
10      <li>Rio de Janeiro
11      <li>Brasilia
12      <li>Salvador
13      <li>Buenos Aires
14      <li>NRA
15    </ol>
16    <b>Qual é a capital da Paraíba?</b>
17    <ol type="A">
18      <li>Maceió
19      <li>Aracaju
20      <li>Fortaleza
21      <li>João Pessoa
22      <li>NRA
23    </ol>
24  </font>
25 </body>
26 </html>

```

Listas Não Ordenadas

- A estrutura das listas não ordenadas é a mesma das listas ordenadas.
- A diferença é que, na apresentação, os **itens serão precedidos por um marcador (bullet)**.
- No caso de existir uma lista dentro de outra, será usado um marcador diferente para os itens de cada lista.

Deve-se substituir o identificador por e por .

Exemplos de Listas Não Ordenadas

- ▶ Tags e listas simples:

```

<ul>
  <li>Um item de lista</li>
  <li>Outro item de lista</li>
</ul>

```

- ▶ que aparecerá assim no seu navegador:

- Um item de lista
- Outro item de lista

Listas Não Ordenadas

- ▶ **Sintaxe:**

```
<ul type="bullet"> <li>item1 <li>item2 ... </ul>
```

- ▶ **Em que: type - tipo de bullet que precede cada item.**

- ▶ **Pode ser do tipo:**

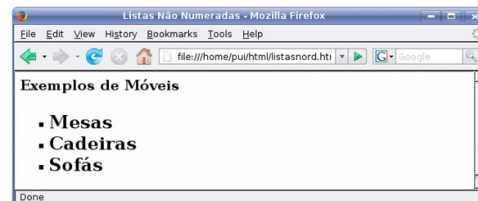
- ▶ disk - pequeno disco sólido
- ▶ square - quadrado preenchido
- ▶ circle - círculo cheio

Exemplos de Listas Não Ordenadas

```

1 <html>
2 <head>
3 <title>Listas Não Numeradas</title>
4 </head>
5 <body>
6 <h3>Exemplos de Móveis</h3> <p>
7 <h2>
8 <ul type="square">
9 <li>Mesas</li>
10 <li>Cadeiras</li>
11 <li>Sofás</li>
12 </ul>
13 </h2>
14 </body>
15 </html>

```



Lista de Definições

- Usada em situações em que termos da lista necessitam de definições, como em dicionários ou glossários.
- Estas listas têm dois níveis de informações.
 - O primeiro é o **tópico**, que aparece em destaque.
 - O segundo é a **descrição** que aparecerá deslocada em relação ao tópico.
- Ex.:

Termo de definição em uma lista de definições (DL).

```

item 1
item 2
item 3
item 4

```

- São iniciadas com o identificador **<dl>** e encerradas com **</dl>**.
- Cada item tem seu tópico e sua descrição, iniciados por **<dt>** e **<dd>** respectivamente.
- Sintaxe:

<dl> <dt>tópico <dd>descrição </dl>

Exemplos de listas de definições:

```

1 <html>
2   <head>
3     <title>Listas de Definições</title>
4   </head>
5   <body>
6     <dl>
7       <dt>Gentle
8       <dd>dócil, manso, brando, tolerante, <br>
9         pacato, pacífico, <br>
10        benévolo, propício, tranqüilo, gentil, amável, de boa estirpe, nobre.
11      <p>
12    <dt>Gentlefolk
13    <dd>nobreza, fidalguia, burguesia. <p>
14    <dt>Gentleman
15    <dd>cavalheiro.
16  </dl>
17 </body>
18 </html>

```

5. Tags de Tabelas

As **tabelas**, tal como as listas, **são um sistema de elementos HTML (ou tags) usados para criar um papel especialmente formatado.**

Atualmente a grande maioria das páginas na Web utilizam esse recurso. A estrutura do comando para inserir uma tabela é a seguinte: **<TABLE atributos>... </TABLE>**.

Para informar onde começa e termina cada linha usamos o comando **<TR>... </TR>** (o elemento é TR de “table row”, onde “row” é linha em inglês). Dentro do comando **<TR>**, cada célula é identificada usando o elemento **<TD>... </TD>** (o elemento é TD de “table data”, já que cada célula contém dados da tabela).

Veja no quadro abaixo o exemplo de uma tabela bem simples, com somente **1(uma) linha e 2(duas) colunas**:

```

<TABLE>
<TR>
<TD> Aqui o conteúdo da célula 1 </TD>
<TD> Aqui o conteúdo da célula 2 </TD>
</TR>
</TABLE>

```

<TABLE>... </TABLE> Cria uma tabela.

<TR>... </TR> Cria linhas de tabelas.

<TD>... </TD> Cria uma célula em uma tabela.

6.Tag de Imagens

**** Fornece informações sobre a origem, o posicionamento e o comportamento da imagem. Esta tag posiciona a imagem na página.

Atributos de tags

As tags podem conter atributos, que fornecem informações adicionais sobre o elemento, como class, id, name, src (para imagens e scripts) e href (para links). O atributo id identifica um único elemento de forma única dentro da página, enquanto name é usado principalmente em formulários para associar valores a campos e permitir sua submissão ao servidor.

Os atributos HTML se dividem em quatro tipos:

- Globais: São aplicáveis a qualquer tag tais como id, class, style title. Dessas destacamos o atributo id que indica um ID para uma tag. CUIDADO!!! Infelizmente não é gerado nenhum tipo de erro no navegador se duas tags tiverem o mesmo ID.
- Específicos: Dependem da tag e só fazem sentido nela, por exemplo o href só faz sentido na tag <a>
- Booleanos: Não precisam de valor, por exemplo, se eu coloco disabled em uma tag esse é um atributo que informa que ela será mostrada desabilitada no navegador. Exatamente como ela será mostrada (estilo) depende do CSS.
- Eventos: São atributos que dizem o que deve ser feito se um determinado evento ocorrer sobre a tag, por exemplo onClick.

DOM

Conceito

Mais uma vez você deve estar se perguntando: O que DOM tem a ver com JS? Ocorre que várias questões de JS para concursos são de como acessar o DOM utilizando JS. Por isso decidimos explicar um pouco o que é DOM.

O DOM (Document Object Model) é uma interface de programação que representa um documento HTML ou XML como uma estrutura de árvore, onde cada elemento da página é um nó. Um nó pode ser um elemento (como <div> ou <p>), um atributo (como class ou id) ou um texto dentro de uma tag. O DOM permite que JavaScript acesse e modifique dinamicamente os elementos de uma página, alterando seu conteúdo, estilos e estrutura.

Acessando nós no DOM com JavaScript

Existem várias formas de acessar um nó no DOM usando JavaScript:

`document.getElementById("meuElemento")` → Retorna um único elemento pelo seu id. Uma curiosidade, não é gerado nenhum erro pelo navegador se duas tags tenham o mesmo ID. Nesse caso o `getElementById` retornará o primeiro elemento com esse ID dentro da hierarquia, mas CUIDADO!!! Isso não está na especificação e nem em nenhum documento, portanto, não há como garantir que o navegador siga esse padrão.

`document.getElementsByClassName("minhaClasse")` → Retorna uma coleção de elementos com a classe especificada.

`document.getElementsByTagName("p")` → Retorna uma coleção de elementos com a tag especificada.

`document.querySelector(".minhaClasse")` → Retorna o primeiro elemento que corresponde ao seletor CSS fornecido. Nesse exemplo, o ponto é o seletor de classes, portanto `minhaClasse` é a classe CSS `minhaClasse`.

`document.querySelectorAll("div p")` → Retorna todos os elementos que correspondem ao seletor CSS.

Modificando Estilos e Conteúdo com JavaScript

Depois de acessar um nó, podemos modificar seu estilo ou conteúdo de diversas formas:
Alterando estilos:

```
let elemento = document.getElementById("meuElemento");
elemento.style.color = "red"; // Muda a cor do texto para vermelho
elemento.style.backgroundColor = "yellow"; // Altera a cor de fundo
elemento.style.fontSize = "20px"; // Modifica o tamanho da fonte
```

Modificando nós do DOM

```
let elemento = document.getElementById("meuElemento");
elemento.innerHTML = "<strong>Texto em negrito</strong>"; // Insere HTML dentro do elemento
elemento.textContent = "Novo texto"; // Altera apenas o texto, sem interpretar HTML
```

PRINCIPAIS FRAMEWORKS JAVASCRIPT NO CLIENTE

Mais uma vez, o motivo aqui é apenas dar uma pequena explicação sobre essas frameworks famosas de forma a que, se aparecer na sua prova, você tenha uma mínima ideia sobre o que está sendo falado. Não imaginamos questões sobre essas frameworks em uma prova de JS uma vez que isso seria extremamente específico e fora do que foi cobrado no edital. Ou seja, bastante passível de recursos.

Vue JS

O Vue.js é um framework progressivo de JavaScript para a criação de interfaces de usuário e aplicações de página única (SPA). Ele utiliza um sistema reativo baseado em componentes, facilitando a manipulação do DOM de forma eficiente. Com uma curva de aprendizado suave, o Vue permite integração gradual em projetos e oferece recursos como bindings declarativos, diretivas e um poderoso sistema de estado gerenciado.

React

O React.js é uma framework JavaScript para a construção de interfaces de usuário reativas e eficientes. Ele utiliza um modelo baseado em componentes e um DOM virtual

para otimizar renderizações. Criado pelo Facebook (atualmente META), é amplamente usado para desenvolver aplicações web modernas e escaláveis.

Angular

O AngularJS é o “vovozinho” dos frameworks SPA (Single Page Apps), ele é um framework JavaScript desenvolvido pelo Google para a criação de aplicações web dinâmicas. Ele utiliza o conceito de data binding bidirecional e injeção de dependências para facilitar o desenvolvimento. Embora tenha sido amplamente adotado, foi substituído pelo Angular (versões 2+), que trouxe uma arquitetura mais moderna.

Svelte é um *framework* JavaScript moderno para construção de interfaces de usuário reativas. Diferente de React, Vue ou Angular, que utilizam um virtual DOM, o Svelte compila seus componentes em JavaScript puro durante a fase de build, resultando em um código otimizado e com melhor desempenho em tempo de execução.

Principais características do Svelte

- compilação em tempo de *build*: o código é transformado em JavaScript eficiente antes de ser enviado para o navegador;
- sintaxe simples e intuitiva: usa JavaScript puro com um estilo declarativo para manipular o estado;
- reduz a quantidade de código necessária para gerenciar estados e eventos;
- maior desempenho: como não há virtual DOM, as atualizações são mais rápidas e eficientes.

JAVASCRIPT NO SERVIDOR

NODEJS

Introdução, história e evolução

O NodeJS é um marco na história do JS. Foi a partir do Node que o JS pôde ser utilizado no Servidor. Para funcionar, o Node utiliza o “Motor” do V8, que é utilizado para executar JS nos browsers Chromium (projeto open source de onde o Chrome e, mais atualmente, o Edge se baseiam. Mais uma vez você deve estar se perguntando: Meu Edital está pedindo NodeJS? Na verdade o edital é muito aberto ao colocar Javascript, então optamos por falar em detalhes do JS e dar uma “pincelada” em outros tópicos de JS para, pelo menos, você saber do que se trata. Recomendamos que quando todas as outras matérias estiverem bem estudadas, naquele horário de descanso, você ler um pouco mais sobre o NodeJS.

Fundamentos

1. Gerenciadores de pacotes

a) Npm

i. Conceitos: O NPM é o gerenciador de pacotes do Node. Mas o que seria isso? Ambientes modernos de desenvolvimento de software são extremamente complexos. Na verdade, a tecnologia está cada vez mais complexa uma vez que tudo hoje é tecnologia. Para permitir que o desenvolvimento de software se adeque a várias features diferentes tais como, por exemplo: acesso a bancos, acesso a sistemas de autorização, sanitização de dados (para garantir que dados maliciosos não sejam enviados para o seu servidor) ou outros milhões (sim, milhões) de funcionalidades prontas para o desenvolvimento.

Para isso foi criado o conceito de dependência de software. Você pode instalar diferentes dependências (que também chamamos de PACOTES) a depender do que o seu sistema vai precisar. O NPM é o gerenciador de pacotes do NodeJS, ele permite instalar, excluir e atualizar pacotes. Mas o NPM faz mais que isso, veremos mais características (features) do NPM.

ii. O arquivo package.json:

O Arquivo package.json é o responsável por guardar várias informações diferentes do seu projeto Node. Nele você consegue incluir as dependências do seu projeto, dizer qual o nome da aplicação, versão e até mesmo qual a licença dele, dentre outras várias opções.

iii. Principais opções do npm

1. Install

Npm install é utilizado de duas formas. A primeira, com o nome do pacote, serve para você instalar um determinado pacote no seu projeto. Caso você digite npm install (sem nome do pacote), o npm irá instalar todos os pacotes que já constam no package.json

Opções:

a) Save-dev: Salva o pacote apenas para desenvolvimento (não vai para a versão de produção)

b) Instalação global: Com a opção -g o npm instalará o pacote globalmente na sua máquina.

c) Instalando uma versão específica: Se você digitar npm install PACOTE@VERSAO, o npm irá instalar uma versão específica do pacote

2. Executando scripts do package.json:

O package.json tem uma seção especial chamada scripts. Ele permite que você cadastre scripts JS que serão executados com o comando npm, além de permitir que os scripts tenham acesso a todos os pacotes instalados.

Audit: npm audit faz auditoria de segurança nos pacotes instalados.

4. Init: npm init inicia um projeto, criando o seu package.json

6. Config: npm config permite criar configurações do npm, um bom exemplo é a configuração de proxy.

b) Yarn

Apenas para conhecimento, o yarn foi um projeto que pretendia substituir o npm. Ocorre que, há muito tempo atrás, o npm baixava os pacotes em sequencia, o que fazia

ficar muito lento. O yarn baixava eles em paralelo o que fazia uma grande diferença. Hoje o npm já resolveu esse (e outros problemas) o que reduziu as vantagens do yarn.

FERRAMENTAS DO ECOSISTEMA JAVASCRIPT E SUAS UTILIDADES

Muitas questões de prova trazem termos que fazem parte do ecossistema JavaScript sem, no entanto, serem diretamente ligadas à linguagem. O intuito desse capítulo é passar rapidamente por esses termos de forma a dar ao aluno uma visão geral sobre o tema.

O **ecossistema JavaScript** é composto por diversas ferramentas que ajudam os desenvolvedores a criar, testar e otimizar aplicações de maneira eficiente. Essas ferramentas abordam desde a execução do código até a preparação para produção, garantindo modularidade e desempenho. Abaixo, apresentamos algumas das principais ferramentas e suas funções, destacando o Webpack.

As principais ferramentas são:

- **Babel:** Transpilador que converte código moderno (ES6+) em versões compatíveis com navegadores antigos. Isso era necessário, pois os *browsers* não entendiam ou não havia uma padronização para o JS moderno. Importante: a maior parte das novas versões dos browsers já consegue entender os novos Ecma Script.
- **Webpack:** empacotador de módulos que organiza e otimiza arquivos para produção.
- **ESLint:** ferramenta de linting para manter a consistência e evitar erros no código.
- **Prettier:** Formata o código a partir de padrões comumente aceitos na comunidade.
- **Typescript:** Superset do JavaScript que adiciona tipagem estática, tornando o código mais seguro e fácil de manter.
- **Vite:** ferramenta de build e desenvolvimento ultrarrápida, focada em eficiência e compatibilidade com projetos modernos. É um substituto do Webpack.
- **Jest:** *framework* de testes para validar funcionalidades.

BABEL

- **O que é:** o Babel é uma ferramenta que transforma código JavaScript escrito com sintaxe moderna (como arrow functions, let/const ou async/await) em código equivalente em uma versão mais antiga (como ES5), garantindo compatibilidade com navegadores ou ambientes que não suportam as funcionalidades mais recentes.
- **Utilidade:** permite que desenvolvedores usem as últimas funcionalidades da linguagem sem se preocupar com suporte legado, além de suportar plugins para personalizar transformações.

- **Como funciona:** lê o código fonte, converte-o em uma representação intermediária (AST – Abstract Syntax Tree) e gera o código transpilado.
- **Benefícios do ecossistema:** essas ferramentas, incluindo o Babel, trabalham em conjunto para melhorar a produtividade, a qualidade do código e a compatibilidade. O Babel, por exemplo, é frequentemente combinado com Webpack para projetos maiores, garantindo que o código final seja eficiente e universalmente suportado.
- Se quiser aprofundar seus estudos em Babel, acesse:

Site Oficial: babeljs.io

Complementar: [Alura – Manipulação de Array com Babel](#)

WEBPACK

- **O que é:** o Webpack é uma ferramenta de empacotamento (bundler) que pega módulos JavaScript, junto com outros recursos como CSS, imagens e fontes, e os compila em um ou mais arquivos otimizados para uso no navegador.
- **Utilidade:** resolve dependências entre módulos, reduz o tamanho dos arquivos (com minificação) e facilita o uso de ferramentas como Babel ou carregadores de CSS. É essencial para projetos grandes onde os arquivos precisam ser organizados e entregues eficientemente.
- **Como funciona:** analisa um ponto de entrada (ex.: index.js), constrói um grafo de dependências e gera um *bundle* final.
- **Benefícios do ecossistema:** ferramentas como o Webpack, quando combinadas com outras como Babel ou npm, simplificam o desenvolvimento de aplicações complexas, oferecendo modularidade, otimização e compatibilidade. O Webpack, em particular, é amplamente usado em projetos React, Vue ou Angular para gerenciar assets de forma eficiente.
- Se quiser aprofundar seus estudos em Webpack, acesse:

Site Oficial: webpack.js.org

Complementar: [MDN Web Docs – Webpack](#)

ESLINT

- **O que é:** o ESLint é uma ferramenta de linting que analisa o código JavaScript para encontrar erros, inconsistências de estilo e práticas potencialmente problemáticas. Ele é altamente configurável, permitindo definir regras personalizadas ou usar padrões predefinidos.
- **Utilidade:** garante a consistência do código em equipes, melhora a legibilidade e previne bugs comuns, como variáveis não declaradas ou ponto e vírgula faltando.

- **Como funciona:** examina o código com base em um conjunto de regras configuradas e exibe avisos ou erros, podendo até corrigi-los automaticamente.
- **Benefícios do ecossistema:** O processo de Lint permite que o código JS ou TS seja organizado com regras claras, o que facilita a manutenibilidade do código.
- Se quiser aprofundar seus estudos em ESLint, acesse:

Site Oficial: eslint.org

Complementar: [Dev.to](https://dev.to) – Configurando ESLint

PRETTIER

- **O que é:** Prettier é uma ferramenta de formatação de código que aplica um estilo consistente automaticamente, removendo a necessidade de debates sobre convenções como espaçamento ou tamanho de linha. Suporta JavaScript, TypeScript, CSS, HTML e mais.
- **Utilidade:** garante uniformidade no código em equipes, economiza tempo e integra-se bem com ferramentas como ESLint e editores (ex.: VS Code).
- **Como funciona:** analisa o código e o reformata com base em regras predefinidas ou configuráveis, aplicando ajustes como indentação e quebras de linha.
- **Benefícios do Ecossistema:** ferramentas como o Prettier, combinadas com ESLint, Webpack ou Jest, promovem um fluxo de desenvolvimento eficiente e padronizado. O Prettier é especialmente útil em projetos colaborativos, garantindo que o estilo do código permaneça consistente sem esforço manual.
- Se quiser aprofundar seus estudos em Prettier, acesse:

Site Oficial: prettier.io

Complementar: freecodecamp.org

TYPESCRIPT

- **O que é:** TypeScript (TS) é um superconjunto de JavaScript desenvolvido pela Microsoft que adiciona tipagem estática opcional e recursos como interfaces e enums. Ele é compilado para JavaScript puro, permitindo uso em qualquer ambiente JS.
- **Utilidade:** melhora a segurança do código ao detectar erros de tipo em tempo de desenvolvimento, facilita a manutenção em projetos grandes e oferece autocompletar avançado em editores como VS Code.
- **Como funciona:** o compilador TypeScript (tsc) transforma o código.ts em JavaScript compatível, com base nas configurações do arquivo tsconfig.json.
- **Benefícios do ecossistema:** o Typescript adiciona ao projeto a confiabilidade da tipagem estática ao seu projeto. Assim, por exemplo, se uma função recebe um

parâmetro em JS qualquer tipo pode ser passado, enquanto que o TS permite que eu diga exatamente o tipo que a função espera, por exemplo number.

- Se quiser aprofundar seus estudos em Typescript, acesse:

Site Oficial: typescriptlang.org

Complementar: [freeCodeCamp – TypeScript Tutorial](https://freeCodeCamp.org/learn/typescript)

VITE

- **O que é:** Vite é uma ferramenta de construção moderna que oferece um servidor de desenvolvimento extremamente rápido e builds otimizados para produção. Criada por Evan You (autor do Vue.js), ela utiliza módulos ES nativos (ESM) no navegador para evitar empacotamentos desnecessários durante o desenvolvimento.
- **Utilidade:** acelera o tempo de inicialização e atualizações com Hot Module Replacement (HMR), sendo uma alternativa mais leve e eficiente ao Webpack para projetos com React, Vue ou vanilla JS.
- **Como funciona:** durante o desenvolvimento, serve arquivos diretamente como módulos ES; na produção, usa Rollup para criar bundles otimizados.
- **Benefícios do Ecossistema:** o Vite torna o desenvolvimento mais rápido e eficiente. O Vite é especialmente útil em projetos front-end modernos, oferecendo uma experiência de desenvolvimento ágil e builds compactos, sendo adotado em frameworks como Vue e React.
- Se quiser aprofundar seus estudos em Vite, acesse:

Site Oficial: vitejs.dev

Complementar: [DEV Community – Migrando para Vite](https://dev.to/mauricioferreira/migrando-para-vite)

JEST

- **O que é:** Jest é um framework de testes desenvolvido pelo Facebook, projetado para ser simples e poderoso. Ele é amplamente usado para testar código JavaScript, incluindo aplicações React, Node.js e vanilla JS.
- **Utilidade:** permite escrever testes unitários, de integração e snapshots para garantir que o código funcione como esperado, com suporte a asserções, mocks e cobertura de código integrado.
- **Como funciona:** executa arquivos de teste (geralmente com extensão.test.js ou.spec.js) e verifica se os resultados correspondem às expectativas definidas.
- **Benefícios do Ecossistema:** o Jest, assim como outras ferramentas de automatização de teste, garante que novas funcionalidades não irão “quebrar” funcionalidades antigas. Através do teste automatizado de toda a sua base de código é possível garantir que

o seu produto final seja mais estável, aparecendo menos erros em outras partes do sistema.

- Se quiser aprofundar seus estudos em Jest, acesse:

Site Oficial: jestjs.io

Complementar: [Wisdom Geek – Jest com TypeScript](#)

RESUMO

Segue um resumo antes da prova. Ele serve para lembrar os principais tópicos de forma rápida. BOA SORTE!

- **O que é o Javascript:** JavaScript é uma linguagem de programação dinâmica usada principalmente para criar interatividade em páginas web. Ele é amplamente suportado por navegadores e pode ser executado no lado cliente e servidor.
- **Histórico:** Criado pela Netscape em 1995 como LiveScript, foi posteriormente renomeado para JavaScript. Em 1997, passou a ser padronizado pela ECMA como ECMAScript, evoluindo constantemente.
- **Tipagem:** JavaScript possui tipagem fraca e dinâmica, permitindo que variáveis mudem de tipo durante a execução. Isso torna a linguagem flexível, mas pode causar erros inesperados.
- **Estruturas:** Utiliza estruturas como if/else, switch, for, while e do while para controle de fluxo. Além disso, permite o uso de funções, classes e objetos para modularização do código.
- **Assíncrono:** Trabalha com operações não bloqueantes usando Callbacks, Promises e async/await. Isso permite realizar requisições HTTP ou manipular eventos sem travar a execução do código.
- **Cliente:** No lado do navegador, JavaScript interage com o DOM para manipular elementos HTML. Ele também lida com eventos do usuário, animações e interações dinâmicas na interface.
- **Fetch API:** API moderna para realizar requisições HTTP assíncronas de forma mais simples que XMLHttpRequest. Utiliza Promises e permite buscar e enviar dados para servidores.
- **XMLHttpRequest:** Método mais antigo para requisições HTTP assíncronas, usado para AJAX. Ainda é suportado, mas geralmente substituído pelo fetch por ser mais verboso.
- **HTML:** Linguagem de marcação usada para estruturar páginas web. Define elementos como textos, imagens, links e botões que podem ser manipulados via JavaScript.
- **Tags HTML:** São elementos usados para definir a estrutura da página, como <div>, , <p> e <h1>. Elas podem conter atributos que alteram seu comportamento e aparência.
- **Tags Iniciais:** Incluem <html>, <head> e <body>, que são essenciais para qualquer documento HTML. O <head> contém metadados, enquanto <body> contém o conteúdo visível.

- **Tags de Formatação:** Como `<b>`, `<i>`, `<u>`, `<strong>` e `<em>`, são usadas para estilizar textos. Algumas indicam significado semântico, como `<strong>` (ênfase) e `<em>` (destaque).
- **Tags de Links:** A `<a>` cria links para outras páginas ou seções internas. Pode usar atributos como `href` para definir o destino e `target` para abrir em uma nova aba.
- **Tags de Lista:** `<ul>` (lista não ordenada), `<ol>` (lista ordenada) e `<li>` (item da lista). São usadas para organizar informações em listas com marcadores ou numeração.
- **Tags de Tabelas:** `<table>` define uma tabela, `<tr>` representa linhas, `<td>` células e `<th>` cabeçalhos. Permitem organizar dados de forma estruturada.
- **Tag de Imagens:** `<img>` exibe imagens e utiliza o atributo `src` para definir o arquivo. Pode conter `alt` para acessibilidade e `width` e `height` para tamanho.
- **Atributos de TAGS HTML:**
 - **Globais:** Aplicáveis a qualquer tag, como `id`, `class`, `style`, `title` e `data-*`.
 - **Específicos:** Dependem da tag, como `href` para `<a>`, `src` para `<img>` e `type` para `<input>`.
 - **Booleanos:** Não precisam de valor, como `disabled`, `readonly` e `checked`.
 - **Eventos:** Usados para capturar ações do usuário, como `onclick`, `onchange` e `onmouseover`.
- **DOM:** Representação em árvore da estrutura HTML, onde cada elemento é um nó manipulável via JavaScript. Métodos como `getElementById` e `querySelector` permitem acessar e modificar elementos.
- **Frameworks:** React, Angular e Vue são bibliotecas/frameworks que facilitam a criação de aplicações interativas. Eles oferecem componentes reutilizáveis e gerenciamento de estado eficiente.
- **Servidor:** JavaScript também pode ser usado no back-end com Node.js. Isso permite criar APIs, manipular bancos de dados e desenvolver aplicações completas.
- **Babel:** Transpila código moderno para versões compatíveis com navegadores antigos. Isso permite usar as últimas funcionalidades da linguagem sem se preocupar com compatibilidade.
- **Webpack:** Um bundler que combina e otimiza arquivos CSS, JS e imagens para melhorar o desempenho. Ele permite dividir código em módulos e carregá-los de forma eficiente.
- **ESLint:** Ferramenta que analisa código para garantir boas práticas e evitar erros comuns. Ajuda a manter um código limpo e padronizado dentro de equipes.
- **Prettier:** Automatiza a formatação do código para manter um estilo consistente. Suporta múltiplas linguagens e pode ser integrado com editores como VSCode.

- **Typescript:** Superset do JavaScript que adiciona tipagem estática. Ajuda a evitar erros e torna o código mais seguro e fácil de manter.
- **Vite:** Ferramenta moderna para desenvolvimento front-end, mais rápida que Webpack. Utiliza ES Modules para carregar arquivos de forma eficiente no navegador.
- **Jest:** Framework de testes para JavaScript, usado para testar funções e componentes. Suporta testes assíncronos e mocks para simular comportamentos.
- **JSON:** Formato leve de intercâmbio de dados baseado em chave-valor, semelhante a objetos JavaScript. É amplamente usado para comunicação entre sistemas e armazenamento de dados estruturados.
- **NodeJS:** Ambiente de execução JavaScript no servidor baseado no motor V8 do Chrome. Permite criar APIs, manipular arquivos, conectar a bancos de dados e executar scripts fora do navegador.
- **NPM:** Gerenciador de pacotes do Node.js, usado para instalar, atualizar e remover bibliotecas. Possui um repositório online de pacotes e facilita a gestão de dependências em projetos.
- **Principais parâmetros do NPM:**
 - npm install [pacote] – Instala um pacote no projeto.
 - npm update – Atualiza dependências para a versão mais recente compatível.
 - npm uninstall [pacote] – Remove um pacote do projeto.
 - npm init – Cria um arquivo package.json com as configurações do projeto.
 - npm run [script] – Executa scripts definidos no package.json.
- **YARN:** Alternativa ao NPM, otimizada para velocidade e eficiência no gerenciamento de pacotes. Suporta caching offline e paralelismo para instalações mais rápidas.
- **Angular:** Framework completo baseado em TypeScript para criar aplicações SPA. Possui estrutura modular, injeção de dependências e binding bidirecional entre HTML e dados.
- **React:** Biblioteca JavaScript para construção de interfaces dinâmicas baseadas em componentes. Utiliza um Virtual DOM para melhorar o desempenho e tem forte integração com o ecossistema JavaScript.
- **VueJS:** Framework progressivo para construção de interfaces interativas e SPA. Oferece reatividade declarativa, fácil aprendizado e boa integração com projetos existentes.
- **DOM:** Representação estruturada da página HTML como uma árvore de elementos manipuláveis por JavaScript. Permite alterar dinamicamente o conteúdo, estilos e estrutura da página.
- **Principais comandos JS para acessar o DOM:**
 - document.getElementById(id) – Retorna um elemento pelo ID.

- `document.querySelector(seletor)` – Retorna o primeiro elemento que corresponde ao seletor CSS.
- `document.querySelectorAll(seletor)` – Retorna uma `NodeList` de elementos correspondentes ao seletor.
- `document.getElementsByClassName(classe)` – Retorna uma coleção de elementos por classe.
- `document.getElementsByTagName(tag)` – Retorna uma coleção de elementos por nome da tag.

QUESTÕES DE CONCURSO

001. (FCC/TRT – 6ª REGIÃO (PE)/ANALISTA JUDICIÁRIO/ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE: TECNOLOGIA DA INFORMAÇÃO/2025) Um Analista precisa buscar em um código JavaScript, o primeiro elemento no DOM em uma página HTML que tenha a propriedade id configurada com o valor imagens. Para isso, ele pode utilizar a instrução:

- a) `document.querySelector('#imagens');`
- b) `document.getElementFromId('imagens');`
- c) `document.querySelector('.imagens');`
- d) `document.getElementById('#imagens');`
- e) `document.getIdentification('#imagens');`

002. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL – TECNOLOGIA DA INFORMAÇÃO/ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) Assinale a opção que corresponde ao comando usado para fazer um comentário de uma linha em JavaScript.

- a) `**`
- b) `%%`
- c) `//`
- d) `&&`
- e) `""`

003. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL/TECNOLOGIA DA INFORMAÇÃO – ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) A diferença entre o armazenamento `localStorage` e `sessionStorage` em JavaScript é a seguinte:

- a) `localStorage` tem uma capacidade de armazenamento maior que `sessionStorage`.
- b) `localStorage` persiste dados mesmo após o navegador ser fechado, enquanto `sessionStorage` limpa os dados quando a sessão do navegador é encerrada.
- c) `localStorage` só pode armazenar strings, enquanto `sessionStorage` pode armazenar qualquer tipo de dado ou objeto.
- d) `localStorage` é mais seguro e recomendado para armazenar informações sensíveis.
- e) `sessionStorage` compartilha os dados entre diferentes abas e janelas, enquanto `localStorage` não.

004. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL/TECNOLOGIA DA INFORMAÇÃO/ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) A API JavaScript que é comumente usada para fazer uma requisição HTTP é

- a) `POSTRequest`
- b) `APIRequest`.
- c) `FetchAPI`.

- d) HttpReq.
- e) WebAPI.

005. (QUADRIX/CFO/ANALISTA DE DESENVOLVIMENTO/2025) Os desenvolvedores dispõem de uma grande variedade de linguagens de programação, cada uma com suas vantagens e desvantagens. Com base nessa informação, julgue o item a seguir.

O uso de JavaScript em desenvolvimento *web* é restrito ao lado do servidor, não sendo utilizado no lado do cliente.

006. (QUADRIX/CFO/ANALISTA DE DESENVOLVIMENTO/2025) Os desenvolvedores dispõem de uma grande variedade de linguagens de programação, cada uma com suas vantagens e desvantagens. Com base nessa informação, julgue o item a seguir.

JavaScript é uma linguagem essencial no desenvolvimento *web* moderno, sendo amplamente utilizada no lado do cliente para manipulação do DOM, criação de interatividade com o usuário, e integração com APIs externas por meio de chamadas AJAX ou Fetch, além de possibilitar o desenvolvimento *full-stack* com

007. (FUNDATEC/PREFEITURA DE ARATIBA-RS/TÉCNICO EM TI/2025) Analise as seguintes asserções e a relação proposta entre elas:

I – O JavaScript permite a criação de páginas interativas e dinâmicas e é amplamente utilizado para desenvolvimento *web*.

PORQUE

II – O HTML é uma linguagem de marcação responsável por estruturar o conteúdo das páginas *web*, mas não tem capacidade de realizar operações lógicas e interativas.

A respeito dessas asserções, assinale a alternativa correta.

- a) As asserções I e II são proposições verdadeiras, e a II é uma justificativa da I.
- b) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa da I.
- c) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- e) As asserções I e II são proposições falsas.

008. (FGV/TCE-RR/ANALISTA ADMINISTRATIVO/TI, COM ESPECIALIDADE EM DESENVOLVIMENTO DE SISTEMAS/2025) No ecossistema JavaScript existe uma ferramenta que atua como um compilador, permitindo utilizar funcionalidades recentes da linguagem (ex: *async/await*, classes, arrow functions e JSX), convertendo código ECMAScript 2015+ em uma versão compatível com navegadores mais antigos. Tal ferramenta é denominada

- a) Express.js.
- b) npm.

- c) Gulp.
- d) webpack.
- e) Babel.

009. (FGV/TCE-RR/ANALISTA ADMINISTRATIVO/TI, COM ESPECIALIDADE EM DESENVOLVIMENTO DE SISTEMAS/2025) Angular, React e Vue.js são poderosas ferramentas para o desenvolvimento de interfaces de usuários *Single Page Applications* e sistemas robustos. Elas apresentam, porém, diferentes abordagens.

Diante desse cenário, avalie as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

- () Nuxt é um dos principais frameworks baseados em Angular.
- () Uma característica do React é o emprego de JSX – extensão de sintaxe que permite escrever marcações semelhantes à HTML em um arquivo JavaScript.
- () Mantido pelo Google, o Vue.js é baseado em TypeScript e no padrão MVC.

As afirmativas são, respectivamente,

- a) F – F – F.
- b) F – V – V.
- c) F – V – F.
- d) V – F – V.
- e) V – V – F.

010. (FCC/TRT-20R(SE)/ANALISTA JUDICIÁRIO/ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE TI/2024) Considere a função abaixo em um arquivo JavaScript de uma aplicação web, implementada em condições ideais.

```
function menu() {  
    var lks = document.getElementById("links");  
    lks.classList.toggle("visible");  
}
```

- a) faz com que o elemento identificado como links tenha a classe visible adicionada ou removida cada vez que a função menu () for chamada.
- b) torna visível a lista de classes CSS que o elemento contido na variável lks possui, toda vez que a função menu () for chamada.
- c) não será executada, pois a linha anterior possui erro na escrita do método getElementById, interrompendo a execução da função.
- d) aplica o valor visible ao elemento cujo id possui o valor links, tomando-o visível na página.
- e) gerará um erro, pois a função toggle não pode ser aplicada no atributo class do elemento HTML cujo id contém o valor links.

011. (IBFC/TRF-5/ANALISTA JUDICIÁRIO-ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE ANÁLISE DE DADOS/2024) JSON (JavaScript Object Notation – Notação de Objetos JavaScript) é uma formatação leve de troca de dados. JSON é em formato texto e completamente independente de linguagem, pois usa convenções que são familiares às linguagens C e familiares, incluindo C++, C#, Java, JavaScript, Perl, Python e muitas outras. Estas propriedades fazem com que JSON seja um formato ideal de troca de dados. Assinale a alternativa que apresenta as duas estruturas em que um JSON é constituído.

- a) Estrutura 1. Uma coleção de pares valor/nome
- b) Estrutura 2. Uma lista duplamente ligada
- c) Estrutura 1. Uma coleção de pares nome/valor Estrutura 2. Uma classe com atributos abstratos
- d) Estrutura 1. Uma coleção de pares nome/valor Estrutura 2. Uma lista ordenada de valores
- e) Estrutura 1. Uma coleção de pares valor/nome Estrutura 2. Uma lista circular

012. (FCC/TRT-20R(SE)/TÉCNICO JUDICIÁRIO-ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE TI/2024) Uma Técnica de um Tribunal do Trabalho usou o seguinte código exemplo JSON para converter um objeto JavaScript em uma string JSON:

```
const object = { nome: "Renata", idade: 25 };
const jsonString =     I    ;
console.log(jsonString); // Saída: {"nome":"Renata","idade":25}
```

O código estará correto se a lacuna 1 for corretamente preenchida por

- a) JSON.encode (object)
- b) JSON.stringify(object)
- c) stringify.JSON (object)
- d) object.stringify()
- e) object.toJSONString()

013. (FCPC/UFC/ANALISTA DE TI/ÁREA: ARQUITETURA E DESENVOLVIMENTO DE SISTEMAS – BACK-END/2025) Considere o seguinte código JavaScript:

```
let x = 10;
const y = 20;
const somar = (a, b) => {
    let resultado = a + b;
    return resultado;
};
x = 15;
y = 25;
console.log(somar(x, y));
```

Qual será o resultado ao executar o código acima?

- a) Uncaught TypeError: invalid assignment to var 'x'.
- b) Uncaught TypeError: invalid assignment to const 'y'.
- c) 40.
- d) resultado 40.

014. (FCPC/UFC/TÉCNICO DE TI/ÁREA: DESENVOLVIMENTO DE SOFTWARE – FRONT-END/2025)

Analise o seguinte trecho de código escrito em JavaScript:

```
const numeros = [3, 2, 4, 1, 5];  
const resultado = numeros.reduce((x, y) => x * y, 1);  
console.log(resultado);
```

Qual será a saída exibida no console?

- a) 3
- b) 6
- c) 120
- d) 150

015. (FCPC/UFC/TÉCNICO DE TI/ÁREA: DESENVOLVIMENTO DE SOFTWARE – FRONT-END/2025)

Qual método de array em JavaScript retorna um novo array com todos os elementos que passam em um teste (função fornecida), sem alterar o array original?

- a) forEach()
- b) filter()
- c) push()
- d) find()

016. (FCPC/UFC/TÉCNICO DE TI/ÁREA: DESENVOLVIMENTO DE SOFTWARE – FRONT-END/2025)

Considere o seguinte código escrito em JavaScript:

```
let numA = 100;  
let numB = "100";  
console.log(numA == numB);  
console.log(numA === numB);
```

Qual será a saída no console?

- a) true e true
- b) true e false
- c) false e true
- d) false e false

017. (FCPC/UFC/ANALISTA DE TI/ÁREA: ARQUITETURA E DESENVOLVIMENTO DE SISTEMAS – BACK-END/2025) Sobre as características do NodeJS, marque a alternativa correta.

- a) O NodeJS foi construído para ser utilizado em servidores Linux e Windows, não fornecendo suporte ao sistema operacional MacOS.
- b) Por se basear na linguagem JavaScript, somente é possível criar aplicações Web com o NodeJS. Entretanto, fornece a escalabilidade necessária em sistemas Web.
- c) O NodeJS é um ambiente de execução JavaScript que roda do lado servidor da comunicação, possibilitando a geração de páginas dinâmicas antes de serem enviadas aos clientes.
- d) Uma das características inovadoras do NodeJS é a capacidade de possibilitar uma comunicação Web síncrona, aguardando finalizar o atendimento de uma solicitação para tratar a seguinte.

018. (FCPC/UFC/TÉCNICO DE TI/ÁREA: DESENVOLVIMENTO DE SOFTWARE – BACK-END/2025) Marque a alternativa que contém o nome do recurso existente na linguagem JavaScript que possibilita o alinhamento da declaração de funções, onde a função mais interna possui acesso aos recursos definidos na função externa?

- a) Sorted.
- b) Closures.
- c) Promises.
- d) WeakMap.

019. (CESPE-CEBRASPE/PC-DF/GESTOR DE APOIO ÀS ATIVIDADES POLICIAIS CIVIS/ ESPECIALIDADE: ANALISTA DE INFORMÁTICA: DESENVOLVIMENTO DE SISTEMAS/2025) Julgue o próximo item, a respeito de desenvolvimento de sistemas.

Na execução do trecho de código a seguir, escrito em JavaScript, o resultado lógico da operação `x == 7` será falso.

020. (IMPARH/CGM DE FORTALEZA-CE/AUDITOR DE CONTROLE INTERNO – ÁREA 2 (CIÊNCIAS DA COMPUTAÇÃO)/2025) Considere as afirmações abaixo e marque a opção correta.

I – O comando `break` pode ser usado para interromper um `loop for` ou `while` em JavaScript. Esse comando permite que o `loop` termine antes que sua condição final seja atingida.

PORQUE

II – Quando o `break` é utilizado em um `loop`, ele encerra a iteração atual e passa para a próxima, mantendo o `loop` em execução.

- a) As duas afirmações são verdadeiras e a segunda justifica a primeira.
- b) As duas afirmações são verdadeiras, mas a segunda não justifica a primeira.
- c) A primeira afirmação é verdadeira e a segunda é falsa.
- d) A primeira afirmação é falsa e a segunda é verdadeira.

021. (INSTITUTO ACESSO/CÂMARA DE MANAUS-AM/ANALISTA DE SEGURANÇA DA INFORMAÇÃO/2024) TypeScript é compilado para JavaScript, o que significa que o código TypeScript é transformado em código JavaScript que pode ser executado em qualquer navegador ou ambiente que suporte JavaScript. O código TypeScript ao ser compilado, se tornaria, por exemplo, a isso em JavaScript:

```
function saudacao(nome: string) {  
  return `Olá, ${nome}!`;  
}
```

```
console.log(saudacao("Mundo TypeScript"));
```

a) `function saudacao(nome:string) { return "Olá, " + nome + "!"; } console.log(saudacao("Mundo TypeScript"));`

b) `function saudacao(string) { return "Olá, " + nome + "!"; } console.log(saudacao("Mundo TypeScript"));`

c) `function saudacao(nome) { return "Olá, " + nome + "!"; } console.log(saudacao("Mundo TypeScript"));`

d) `function saudacao(nome) { return "Olá, " + nome + "!"; } console.log(saudação{Mundo TypeScript});`

e) `function saudacao(nome) { return `Olá, ${nome}!`; } console.log(saudacao("Mundo TypeScript"));`

022. (INSTITUTO ACESSO/CÂMARA DE MANAUS-AM/ANALISTA DE SISTEMAS/2024) Webpack e NPM são ferramentas essenciais no desenvolvimento de aplicações modernas em JavaScript. Enquanto NPM é usado para gerenciar pacotes e dependências, o Webpack ajuda a organizar e empacotar os arquivos. Qual das alternativas abaixo representa corretamente uma funcionalidade do Webpack?

a) Webpack permite modularizar e empacotar arquivos de código, incluindo CSS e imagens, facilitando o carregamento otimizado no navegador.

b) Webpack é uma ferramenta de gerenciamento de pacotes que instala dependências no projeto, como bibliotecas externas e frameworks.

Webpack é responsável pela execução do código JavaScript no lado do servidor, substituindo o Node.js.

c) Webpack gera um arquivo package.json que armazena as dependências do projeto e versões específicas.

d) Webpack serve para compilar apenas código JavaScript, sem suportar a inclusão de arquivos CSS ou imagens.

023. (CESPE-CEBRASPE/TSE/TÉCNICO JUDICIÁRIO/ÁREA: APOIO ESPECIALIZADO – ESPECIALIDADE: PROGRAMAÇÃO DE SISTEMAS/2024) Julgue o próximo item, relativos a linguagens e tecnologias de programação.

O código a seguir, escrito em HTML e JavaScript, ao ser executado, apresentará 2 como resultado.

```
<html>
<body>
<p id="cpnuje"></p>
<script>
for (let i = 0; i < 10; ++i) {
if (i === 3) { break; }
document.getElementById("cpnuje").innerHTML
= i;
}
</script>
</body>
</html>
```

024. (FUNDATEC/SAAE/VIÇOSA-MG/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2024) Analise o código HTML abaixo, o qual contém programação na linguagem JavaScript.

```
<html>
<body>
<h2>Minha primeira Web Page</h2>
<button type="button" name="Tente" onclick="document.write('>Clicou>')">Tente</button>
</body>
</html>
```

O que será exibido ao clicar no botão “Tente”?

- a) >Clicou>
- b) >Clicou>Tente
- c) Tente
- d) Minha primeira Web Page>Clicou>
- e) Minha primeira Web Page>Clicou>Tente

025. (FUNDATEC/IF-AP/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2024) “Na linguagem JavaScript, o texto entre _____ e o final de uma linha é tratado como comentário”. Assinale a alternativa que preenche corretamente a lacuna do trecho acima.

- a) //
- b) \\
- c) %%

- d) --
- e) ??

026. (FGV/TJ-RR/ANALISTA JUDICIÁRIO/DESENVOLVIMENTO DE SISTEMAS/2024) Em muitas linguagens de programação, o bloco try-catch é uma estrutura fundamental para o tratamento de exceções.

No Javascript, para tratar múltiplas exceções dentro de um mesmo bloco try-catch, podemos

- a) utilizar múltiplos blocos catch após o mesmo bloco try, cada um para uma exceção específica.
- b) especificar os diferentes tipos de exceção na cláusula catch, separados por vírgulas.
- c) utilizar o bloco finally para capturar exceções adicionais não tratadas pelo bloco catch.
- d) usar um único bloco catch e, dentro dele, diferenciar as exceções usando estruturas condicionais como if ou switch.
- e) declarar os tipos de exceção diretamente no bloco try usando parâmetros específicos.

027. (UFES/UFES/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2024) Em um projeto de desenvolvimento *web*, é necessário que, ao clicar um botão, o texto de um elemento *div* seja atualizado para “Texto Atualizado”.

Considerando as boas práticas em JavaScript, a abordagem adequada para implementar a funcionalidade de atualizar o texto conforme descrito é:

- a) Alterar o atributo *value* do *div* para “Texto Atualizado” e definir um onclick diretamente no elemento *button* para executar a alteração.
- b) Atualizar o texto do *div* diretamente na função de carregamento da página, sem adicionar um manipulador de eventos ao botão, para definir o texto inicial.
- c) Modificar o estilo do *div* para mostrar o texto “Texto Atualizado” e adicionar um manipulador de eventos *onClick* ao botão, utilizando o estilo para esconder e exibir o texto.
- d) Usar a propriedade *textContent* para modificar o texto do *div* e configurar a alteração do texto utilizando uma função anônima diretamente na propriedade *onclick* do botão.
- e) Utilizar a propriedade *innerHTML* para alterar o conteúdo do *div* e adicionar um *eventListener* ao botão, criando uma função de manipulador de eventos separada, para garantir a modularidade e a manutenção do código.

028. (FUNDATEC/PREFEITURA DE CRUZ ALTA – RS/ANALISTA DE SISTEMAS/2024) Na linguagem JavaScript, qual das instruções abaixo é utilizada para interromper a execução de uma função e retornar um valor para a instrução que invocou essa função?

- a) stop
- b) return
- c) break

- d) go back
- e) exit

029. (QUADRIX/CRT-MG/GESTOR DE TI/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Uma das desvantagens do CSS3 é que ele não permite especificar a quantidade de espaço entre as linhas, o espaçamento entre caracteres ou as margens da página.

030. (QUADRIX/CRT-MG/GESTOR DE TECNOLOGIA DA INFORMAÇÃO/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Os atributos globais são aqueles que podem ser usados somente com os elementos novos da HTML 5, e não com todos os seus elementos.

031. (QUADRIX/CRT-MG/GESTOR DE TECNOLOGIA DA INFORMAÇÃO/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Em HTML5, o atributo *src* fornece as especificidades da imagem que o programador deseja incluir.

032. (QUADRIX/CRT-MG/GESTOR DE TECNOLOGIA DA INFORMAÇÃO/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Em JavaScript, a instrução *do...while* repete a execução de um bloco de código, enquanto uma condição for satisfeita, mas o executa pelo menos uma vez, mesmo que a condição nunca seja satisfeita.

033. (QUADRIX/CRT-MG/GESTOR DE TECNOLOGIA DA INFORMAÇÃO/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Na linguagem de programação JavaScript, o evento *blur* é executado sempre que um documento é abandonado.

034. (IF-SP/IF-SP/PROFESSOR EBTT – INFORMÁTICA/2024) Frameworks de desenvolvimento WEB possuem bibliotecas, padrões de componentes e de design que podem ser utilizados pelos desenvolvedores para facilitar e acelerar a criação de aplicativos. Marque a alternativa correta sobre os frameworks apresentados a seguir e a linguagem na qual eles são baseados.

- a) React é um framework de desenvolvimento WEB baseado em JavaScript.
- b) Django é um framework de desenvolvimento WEB baseado em PHP.
- c) Laravel é um framework de desenvolvimento WEB baseado em JavaScript.
- d) AngularJS é um framework de desenvolvimento WEB baseado em Python.

035. (FGV/SEDUC-SP/PROFESSOR DE ENSINO FUNDAMENTAL E MÉDIO (EDUCAÇÃO PROFISSIONAL) – TECNOLOGIA DA INFORMAÇÃO/2024) Em JavaScript, a palavra-chave _____ é usada para declarar variáveis cujo valor pode mudar no escopo do bloco em que foram definidas, enquanto a palavra-chave _____ é usada para variáveis imutáveis. Já o método _____ adiciona elementos ao final de um array e a estrutura _____ permite a repetição de um bloco de código enquanto uma condição for verdadeira, já o objeto _____ fornece métodos para manipulação de números e cálculos matemáticos.

Em sequência, as palavras-chave que completam corretamente essas lacunas são:

- a) const, let, while, push, Math.
- b) while, push, Math, const, let.
- c) let, const, push, while, Math.
- d) Math, const, let, while, push.
- e) while, Math, const, push, let.

036. (PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Ao trabalhar com padrões web em JavaScript, você deseja criar um script que seja executado apenas após a carga completa da página. Qual evento garante que o script seja executado no momento adequado?

- a) 'DOMContentLoaded'
- b) 'load'
- c) 'onload'
- d) 'documentReady'

037. (PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Na escolha de um framework JavaScript para um novo projeto front-end, é essencial considerar as características únicas que cada um oferece para atender às necessidades específicas do projeto. Ao analisar opções para um projeto front-end, qual característica distingue o Svelte de outros frameworks JavaScript, como Angular, React ou Vue?

- a) Virtual DOM.
- b) Compilação em tempo de construção.
- c) Two-way databinding.
- d) Uso de JSX.

038. (UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Ao implementar um novo recurso em um site com

JavaScript, é preciso armazenar uma coleção de valores. Qual estrutura de dados é mais adequada para esse propósito?

- a) String.
- b) Boolean.
- c) Array.
- d) Character.

039. (PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS/ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Durante a codificação de um script em JavaScript, foi necessário declarar uma variável cujo valor pode mudar ao longo do tempo dentro de um bloco específico. Qual palavra-chave utilizar nesse caso?

- a) 'const'
- b) 'let'
- c) 'var'
- d) 'static'

040. (CESPE/CEBRASPE/TCE-AC/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO – ÁREA: PROJETOS DE TI/2024) Em relação ao desenvolvimento de sistemas web, julgue o próximo item. Nas aplicações SPA (single page application) que utilizam AJAX, cada interação do usuário resulta em um recarregamento completo da página, o que garante que todas as partes da interface sejam atualizadas simultaneamente.

041. (CESPE-CEBRASPE/TCE-AC/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO – ÁREA: PROJETOS DE TI/2024) Acerca de desenvolvimento web e mobile, julgue o item seguinte. O objetivo do JavaScript é deixar mais dinâmicas as aplicações web, de maneira que o usuário possa interagir e alterar o conteúdo da página.

042. (FGV/TRF – 1ª REGIÃO/TÉCNICO JUDICIÁRIO – ÁREA ADMINISTRATIVA – ESPECIALIDADE: DESENVOLVIMENTO DE SISTEMAS DE INFORMAÇÃO/2024) O analista Cléber está implementando um script para automatizar o build de uma aplicação apoiada pelo npm. Para se certificar de que, a cada novo build, o npm reinstalará todas as dependências da aplicação sem alterar o arquivo package.json, Cléber empregou no script de automação o recurso de instalação limpa do npm.

Para solicitar ao npm uma instalação limpa do projeto, Cléber utilizou no script o comando npm:

- a) ci;
- b) dedupe;
- c) shrinkwrap;

- d) `install --clear;`
- e) `install --erase.`

043. (FGV/BBTS/2023) Os tipos de dados suportados pela linguagem ECMAScript (versão 2021) são

- a) Null, Boolean, Binary, Symbol, Real e Integers.
- b) Null, Boolean, Varchar, Symbol, Real, Integers e Object.
- c) Undefined, Null, Boolean, String, Symbol, Number, BigInt e Object.
- d) Undefined, Null, Boolean, Varchar, Symbol, BigInt, Real, Money e Polimorphic.
- e) Complex, Null, Boolean, Binary, Varchar, LongInt, Real, Money, Object e Polimorphic

044. (FGV/TJ RN/2023) No contexto da linguagem JavaScript, analise as afirmativas sobre variáveis declaradas como `const`.

- I – Devem ter seus valores atribuídos na declaração.
- II – Podem ter seus valores alterados depois de declaradas.
- III – Têm sempre escopo global.

Está correto o que se afirma em:

- a) somente II e III;
- b) somente I;
- c) somente II;
- d) somente III;
- e) I, II e III.

045. (FGV/SEF MG/2023) Considere as seguintes expressões usando a linguagem javascript:

`1 == '1'` e `1 === '1'`

Os resultados são, respectivamente,

- a) 1 e 1
- b) true e true
- c) '1' e '1'
- d) false e false
- e) true e false

046. (FGV/TJ RJ/2023) No contexto de um script JavaScript numa página Web, considere o trecho a seguir.

```
<script>
function teste(x) {
return (x / 2);
}
```

alert (teste);

</script>

É correto afirmar sobre esse script que:

- a) há um erro de sintaxe que impede sua execução;
- b) quando executado, exibe a definição da função teste;
- c) quando executado, exibe o valor 0;
- d) quando executado, exibe o valor undefined;
- e) quando executado, o comando alert não é acionado.

047. (FGV/CÂMARA DOS DEPUTADOS/2023) Analise o pretenso código JavaScript a seguir.

```
const xpto = (a, b) => a + b;
```

Sobre esse trecho, é correto afirmar que esse código:

- a) é válido, porque define uma função que retorna a soma de dois números fornecidos como parâmetros;
- b) não compila devido à ausência de um par dos delimitadores "{" e "}";
- c) não compila porque o símbolo "=>" não é permitido na definição de uma função JavaScript;
- d) não compila porque o símbolo "=>" não é permitido numa declaração const;
- e) pode ser compilado, mas gera erro em tempo de execução quando xpto for invocada.

048. (FCC/TRT 20/2016) Em JavaScript, a função do método indexOf() é

- a) procurar um valor dentro de uma string e retornar a quantidade de ocorrências desse valor, se encontrado.
- b) retornar o índice da última ocorrência do texto especificado em uma string.
- c) traír um texto de uma string e retornar a string com o texto extraído em uma nova string.
- d) trocar um valor especificado por espaço em branco na string.
- e) retornar o índice da primeira ocorrência de um texto específico em uma string.

049. (FGV/SEF MG/2023) Analise o código Javascript a seguir.

```
let a = {x: 1};
```

```
let b = {x: 2};
```

```
let c = {x: 3};
```

```
let arr = [a, b, c];
```

```
arr.forEach((obj) => {
```

```
  obj.x += 1;
```

```
});
```

```
console.log(a.x, b.x, c.x);
```

O resultado exibido ao rodar esse código será

- a) 1 2 3

- b) 2 3 4
- c) 4 5 6
- d) [1, 2, 3]
- e) [2, 3, 4]

050. (FGV/PC AM/2022) Considere o trecho JavaScript exibido a seguir.

```
const a = [1,2,3,4,5];  
const b = a.map(xpto);  
alert(b);  
function xpto (x, y) {  
  return x * y;  
}
```

Assinale o conteúdo exibido na execução do script acima.

- a) 0,2,6,12,20
- b) 1,2,3,4,5
- c) 1,3,5,7,9
- d) 1,4,9,16,25
- e) 2,4,6,8,10

GABARITO

- | | | |
|-------|-------|-------|
| 1. a | 18. b | 35. c |
| 2. c | 19. E | 36. a |
| 3. b | 20. c | 37. b |
| 4. c | 21. e | 38. c |
| 5. E | 22. a | 39. b |
| 6. C | 23. C | 40. E |
| 7. b | 24. a | 41. C |
| 8. e | 25. a | 42. a |
| 9. c | 26. d | 43. c |
| 10. d | 27. e | 44. b |
| 11. a | 28. b | 45. e |
| 12. b | 29. E | 46. b |
| 13. c | 30. E | 47. a |
| 14. c | 31. E | 48. e |
| 15. b | 32. C | 49. b |
| 16. b | 33. E | 50. a |
| 17. c | 34. a | |

GABARITO COMENTADO

001. (FCC/TRT – 6ª REGIÃO (PE)/ANALISTA JUDICIÁRIO/ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE: TECNOLOGIA DA INFORMAÇÃO/2025) Um Analista precisa buscar em um código JavaScript, o primeiro elemento no DOM em uma página HTML que tenha a propriedade id configurada com o valor imagens. Para isso, ele pode utilizar a instrução:

- a) `document.querySelector('#imagens');`
- b) `document.getElementFromId('imagens');`
- c) `document.querySelector(':imagens');`
- d) `document.getElementById('#imagens');`
- e) `document.getIdentification('#imagens');`



A letra d) está errada. Para buscar uma informação no documento, usamos a interface *document* do Javascript que nos permite acessar o **DOM** da página web carregada. Para acessar o elemento com id usamos o **getElementById**.

Lembre-se de que não há como garantir que apenas um elemento tem um id, infelizmente não há um erro no navegador ao carregar a página se mais de um elemento. Nesse caso, o **getElementById** irá retornar o primeiro elemento.

Isso nos faria pensar que a opção correta é a letra D, pois é a única com `getElementById`. Ocorre que essa opção está buscando pelo id `'#imagens'` sendo que o enunciado pede para buscar por `'imagens'`. Para entender mais sobre seletores CSS, recomendamos

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_selectors.

a) Certa. Já a **opção A** faz a busca por seletor CSS. **O símbolo # significa o seletor ID**. Nessa opção a busca será feita pelo id imagens, exatamente o que procuramos!

Quanto às outras opções:

- b) Errada. `getElementFromId` não existe.
- c) Errada. Está buscando por todas as tags que tenham uma class CSS chamada imagens.
- e) Errada. Não existe `document.getIdentification`.

Letra a.

002. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL – TECNOLOGIA DA INFORMAÇÃO/ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) Assinale a opção que corresponde ao comando usado para fazer um comentário de uma linha em JavaScript.

- a) `**`
- b) `%%`

- c) //
- d) &&
- e) ""



Os **comentários** são uma parte essencial de qualquer linguagem de programação, incluindo JavaScript. Eles **permitem que os desenvolvedores adicionem anotações, explicações ou desativem trechos de código sem que isso afete a execução do programa.**

Em JavaScript, os comentários não são interpretados pelo motor da linguagem, ou seja, são ignorados durante a execução.

Eles servem para melhorar a legibilidade do código, documentar funcionalidades e facilitar o trabalho entre programadores.

Em Javascript existem dois tipos principais de comentários:

- **Comentários de Uma Linha**

Sintaxe: usa duas barras (//) no início da linha. Tudo após as barras, até o final da linha, é considerado um comentário.

- **Comentários de Várias Linhas**

Sintaxe: inicia com /* e termina com */. Todo o conteúdo entre esses marcadores é considerado um comentário, mesmo que ocupe várias linhas.

Assim, conforme visto a resposta é bem direta e a alternativa correta é a **letra C.**

Letra c.

003. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL/TECNOLOGIA DA INFORMAÇÃO – ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) A diferença entre o armazenamento localStorage e sessionStorage em JavaScript é a seguinte:

- a) localStorage tem uma capacidade de armazenamento maior que sessionStorage.
- b) localStorage persiste dados mesmo após o navegador ser fechado, enquanto sessionStorage limpa os dados quando a sessão do navegador é encerrada.
- c) localStorage só pode armazenar strings, enquanto sessionStorage pode armazenar qualquer tipo de dado ou objeto.
- d) localStorage é mais seguro e recomendado para armazenar informações sensíveis.
- e) sessionStorage compartilha os dados entre diferentes abas e janelas, enquanto localStorage não.



Enquanto o **localStorage** compartilha os dados da mesma origem, o **session storage** é mais restrito pois os dados de uma mesma origem E aba são compartilhados. Alguns exemplos:

1) dados vindos de www.google.com e que estão em abas diferentes compartilham a mesma localStorage enquanto estão em session Storages separadas.

Além disso, os dados da session storage são guardados apenas durante a validade da seção. Para mais informações:

<https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage>

Como, ao fechar o navegador, a sessão se perde, a **letra B** está correta.

localStorage armazena dados indefinidamente (a menos que sejam explicitamente removidos), enquanto sessionStorage armazena dados apenas durante a sessão do navegador (ou seja, enquanto o navegador está aberto).

Letra b.

004. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL/TECNOLOGIA DA INFORMAÇÃO/ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) A API JavaScript que é comumente usada para fazer uma requisição HTTP é

- a) POSTRequest
- b) APIRequest.
- c) FetchAPI.
- d) HttpReq.
- e) WebAPI.



Vamos analisar cada alternativa:

- a) Errada. POSTRequest: este não é um nome padrão para uma API JavaScript para fazer requisições HTTP. POST é um método HTTP, mas não o nome de uma API.
- b) Errada. APIRequest: este é um termo genérico e não se refere a uma API JavaScript específica para fazer requisições HTTP.
- c) Certa. Outra pergunta de resolução direta. A Api chama-se FetchAPI e o comando dela é o fetch. A Fetch API é a API JavaScript padrão e mais utilizada para fazer requisições HTTP de forma assíncrona. A Fetch API utiliza Promises para lidar com as respostas de forma assíncrona, tornando o código mais fácil de ler e manter.
- d) Errada. HttpReq: este não é um nome padrão para uma API JavaScript. Pode ser uma abreviação de "HTTP Request", mas não corresponde a uma API real.
- e) Errada. WebAPI: é um termo genérico que se refere a várias APIs fornecidas pelos navegadores para interagir com o ambiente web, mas não é uma API específica para fazer requisições HTTP.

Letra c.

005. (QUADRIX/CFO/ANALISTA DE DESENVOLVIMENTO/2025) Os desenvolvedores dispõem de uma grande variedade de linguagens de programação, cada uma com suas vantagens e desvantagens. Com base nessa informação, julgue o item a seguir.

O uso de JavaScript em desenvolvimento *web* é restrito ao lado do servidor, não sendo utilizado no lado do cliente.



JavaScript é uma linguagem de programação que desempenha um papel fundamental tanto no lado do cliente (front-end) quanto no lado do servidor (back-end) do desenvolvimento web, embora seu uso mais tradicional e difundido seja no lado do cliente.

Lado do Cliente (*Front-End*):

- **Interatividade:** JavaScript é amplamente utilizado para adicionar interatividade às páginas web. Ele permite criar elementos dinâmicos, responder a ações do usuário (como cliques e movimentos do mouse), validar formulários, e muito mais.
- **Manipulação do DOM:** JavaScript pode manipular o Document Object Model (DOM) para modificar o conteúdo, a estrutura e o estilo das páginas web em tempo real.
- **Single-Page Applications (SPAs):** Frameworks e bibliotecas como React, Angular e Vue.js utilizam JavaScript para criar SPAs, que oferecem uma experiência de usuário mais fluida e responsiva.
- **Animações e Efeitos Visuais:** JavaScript é usado para criar animações, transições e outros efeitos visuais que melhoram a experiência do usuário.

Lado do Servidor (*Back-End*):

- **Node.js:** O Node.js é um ambiente de execução JavaScript que permite que JavaScript seja usado no lado do servidor. Ele permite criar aplicações web escaláveis, APIs, e outras soluções back-end.
- **Desenvolvimento Full-Stack:** Com o Node.js, os desenvolvedores podem usar JavaScript tanto no front-end quanto no back-end, o que facilita o desenvolvimento full-stack.
- **Aplicações Real-Time:** Node.js é frequentemente usado para criar aplicações em tempo real, como chats e jogos online, devido à sua capacidade de lidar com um grande número de conexões simultâneas.

Conforme vimos, essa assertiva é falsa. O NodeJS permite usar o JS no servidor.

Errado.

006. (QUADRIX/CFO/ANALISTA DE DESENVOLVIMENTO/2025) Os desenvolvedores dispõem de uma grande variedade de linguagens de programação, cada uma com suas vantagens e desvantagens. Com base nessa informação, julgue o item a seguir.

JavaScript é uma linguagem essencial no desenvolvimento *web* moderno, sendo amplamente utilizada no lado do cliente para manipulação do DOM, criação de interatividade com o usuário, e integração com APIs externas por meio de chamadas AJAX ou Fetch, além de possibilitar o desenvolvimento *full-stack* com



Lado do Cliente: JavaScript é a principal linguagem para adicionar interatividade, manipular o DOM (Document Object Model), e integrar APIs externas usando AJAX ou Fetch em navegadores web.

Full-Stack: A tecnologia Node.js permite que os desenvolvedores utilizem JavaScript também no lado do servidor, possibilitando o desenvolvimento full-stack, onde a mesma linguagem é usada tanto no front-end quanto no back-end.

Essa assertiva está correta. O JS pode ser usado para todas as funções elencadas na assertiva.

Certo.

007. (FUNDATEC/PREFEITURA DE ARATIBA-RS/TÉCNICO EM TI/2025) Analise as seguintes asserções e a relação proposta entre elas:

I – O JavaScript permite a criação de páginas interativas e dinâmicas e é amplamente utilizado para desenvolvimento web.

PORQUE

II – O HTML é uma linguagem de marcação responsável por estruturar o conteúdo das páginas web, mas não tem capacidade de realizar operações lógicas e interativas.

A respeito dessas asserções, assinale a alternativa correta.

- a) As asserções I e II são proposições verdadeiras, e a II é uma justificativa da I.
- b) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa da I.
- c) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- e) As asserções I e II são proposições falsas.



As duas asserções são verdadeiras, mas não há relação de causa e efeito entre elas.

Letra b.

008. (FGV/TCE-RR/ANALISTA ADMINISTRATIVO/TI, COM ESPECIALIDADE EM DESENVOLVIMENTO DE SISTEMAS/2025) No ecossistema JavaScript existe uma ferramenta que atua como um compilador, permitindo utilizar funcionalidades recentes da linguagem (ex: *async/await*,

classes, arrow functions e JSX), convertendo código ECMAScript 2015+ em uma versão compatível com navegadores mais antigos. Tal ferramenta é denominada

- a) Express.js.
- b) npm.
- c) Gulp.
- d) webpack.
- e) Babel.



Como vimos no nosso curso, essa ferramenta é o Babel. Apenas um ponto, normalmente, ao invés de falar em “compilação”, falamos em “transpilação”. Mas isso não invalida a questão.

Letra e.

009. (FGV/TCE-RR/ANALISTA ADMINISTRATIVO/TI, COM ESPECIALIDADE EM DESENVOLVIMENTO DE SISTEMAS/2025) Angular, React e Vue.js são poderosas ferramentas para o desenvolvimento de interfaces de usuários *Single Page Applications* e sistemas robustos. Elas apresentam, porém, diferentes abordagens.

Diante desse cenário, avalie as afirmativas a seguir e assinale (V) para a verdadeira e (F) para a falsa.

- () Nuxt é um dos principais frameworks baseados em Angular.
- () Uma característica do React é o emprego de JSX – extensão de sintaxe que permite escrever marcações semelhantes à HTML em um arquivo JavaScript.
- () Mantido pelo Google, o Vue.js é baseado em TypeScript e no padrão MVC.

As afirmativas são, respectivamente,

- a) F – F – F.
- b) F – V – V.
- c) F – V – F.
- d) V – F – V.
- e) V – V – F.



F – O Nuxt é um framework para VueJs, inspirado no NextJS, para React.

V – Está correto, o React utiliza o JSX que serve para escrever HTML utilizando-se JS.

F – O VueJS não é mantido pelo Google. O framework mantido pelo Google é o Angular.

Letra c.

010. (FCC/TRT-20R(SE)/ANALISTA JUDICIÁRIO/ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE TI/2024) Considere a função abaixo em um arquivo JavaScript de uma aplicação web, implementada em condições ideais.

```
function menu() {
    var lks = document.getElementById("links");
    lks.classList.toggle("visible");
}
```

- a) faz com que o elemento identificado como links tenha a classe visible adicionada ou removida cada vez que a função menu () for chamada.
- b) torna visível a lista de classes CSS que o elemento contido na variável lks possui, toda vez que a função menu () for chamada.
- c) não será executada, pois a linha anterior possui erro na escrita do método getElementById, interrompendo a execução da função.
- d) aplica o valor visible ao elemento cujo id possui o valor links, tomando-o visível na página.
- e) gerará um erro, pois a função toggle não pode ser aplicada no atributo class do elemento HTML cujo id contém o valor links.



O código acima busca a lista de atributos de "class" daquele elemento e faz um toggle (muda de true pra false ou vice versa) no atributo visible. Dessa forma, ele poderá torná-lo visível ou invisível. Resposta correta, **letra D**.

Letra d.

011. (IBFC/TRF-5/ANALISTA JUDICIÁRIO-ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE ANÁLISE DE DADOS/2024) JSON (JavaScript Object Notation – Notação de Objetos JavaScript) é uma formatação leve de troca de dados. JSON é em formato texto e completamente independente de linguagem, pois usa convenções que são familiares às linguagens C e familiares, incluindo C++, C#, Java, JavaScript, Perl, Python e muitas outras. Estas propriedades fazem com que JSON seja um formato ideal de troca de dados. Assinale a alternativa que apresenta as duas estruturas em que um JSON é constituído.

- a) Estrutura 1. Uma coleção de pares valor/nome
- b) Estrutura 2. Uma lista duplamente ligada
- c) Estrutura 1. Uma coleção de pares nome/valor Estrutura 2. Uma classe com atributos abstratos
- d) Estrutura 1. Uma coleção de pares nome/valor Estrutura 2. Uma lista ordenada de valores
- e) Estrutura 1. Uma coleção de pares valor/nome Estrutura 2. Uma lista circular



Um JSON é uma lista de valor/nome. Um exemplo de JSON seria

```
{
  "nome": "Patricia",
  "idade": 34,
  "email": "patricia@noemail.com",
}
```

Letra a.

012. (FCC/TRT-20R(SE)/TÉCNICO JUDICIÁRIO-ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE TI/2024) Uma Técnica de um Tribunal do Trabalho usou o seguinte código exemplo JSON para converter um objeto JavaScript em uma string JSON:

```
const object = { nome: "Renata", idade: 25 };
const jsonString =   I  ;
console.log(jsonString); // Saída: {"nome":"Renata","idade":25}
```

O código estará correto se a lacuna 1 for corretamente preenchida por

- a) JSON.encode (object)
- b) JSON.stringify(object)
- c) stringify.JSON (object)
- d) object.stringify()
- e) object.toJSONString()



O Objeto JSON contém dois principais métodos: stringify (que converte de objeto para string) e parse (que converte de string para objeto).

Uma pegadinha que pode ser cobrada é que enquanto o stringify é SÍNCRONO, o parse é ASSÍNCRONO.

Letra b.

013. (FCPC/UFC/ANALISTA DE TI/ÁREA: ARQUITETURA E DESENVOLVIMENTO DE SISTEMAS – BACK-END/2025) Considere o seguinte código JavaScript:

```
let x = 10;
const y = 20;
const somar = (a, b) => {
    let resultado = a + b;
    return resultado;
};
x = 15;
y = 25;
console.log(somar(x, y));
```

Qual será o resultado ao executar o código acima?

- a) Uncaught TypeError: invalid assignment to var 'x'.
- b) Uncaught TypeError: invalid assignment to const 'y'.
- c) 40.
- d) resultado 40.



A linha 25 tenta trocar o valor de uma constante o que gera o erro descrito na **letra C**.

Letra c.

014. (FCPC/UFC/TÉCNICO DE TI/ÁREA: DESENVOLVIMENTO DE SOFTWARE – FRONT-END/2025)

Analise o seguinte trecho de código escrito em JavaScript:

```
const numeros = [3, 2, 4, 1, 5];
const resultado = numeros.reduce((x, y) => x * y, 1);
console.log(resultado);
```

Qual será a saída exibida no console?

- a) 3
- b) 6
- c) 120
- d) 150



O comando reduce funciona iterando sob todo o array e retornando o que há dentro da função. O primeiro parâmetro é o valor do último retorno e o segundo parâmetro é o próximo valor a ser iterado. Como, na primeira interação, não há valor anterior, ele é o segundo parâmetro do reduce.

Vamos usar essa questão como exemplo prático, será feita uma iteração sob todo o array, multiplicando o valor anterior com o valor a ser iterado. Repare que há o segundo parâmetro

com o valor um. Dessa forma, na primeira iteração o valor 3 (primeiro valor do array), será multiplicado por 1. Seguiria-se a seguinte ordem:

$$3 * 1 = 3$$

$$2 * 3 = 6$$

$$6 * 4 = 24$$

$$24 * 5 = 120$$

Letra c.

015. (FCPC/UFC/TÉCNICO DE TI/ÁREA: DESENVOLVIMENTO DE SOFTWARE – FRONT-END/2025)
Qual método de array em JavaScript retorna um novo array com todos os elementos que passam em um teste (função fornecida), sem alterar o array original?

- a) forEach()
- b) filter()
- c) push()
- d) find()



Os métodos filter e find tem a característica e passarem por um teste, filtrando o resultado. A diferença é que o filter retorna um array com o filtro enquanto o find retorna o primeiro elemento em que satisfaça a condição.

Letra b.

016. (FCPC/UFC/TÉCNICO DE TI/ÁREA: DESENVOLVIMENTO DE SOFTWARE – FRONT-END/2025)
Considere o seguinte código escrito em JavaScript:

```
let numA = 100;
let numB = "100";
console.log(numA == numB);
console.log(numA === numB);
```

Qual será a saída no console?

- a) true e true
- b) true e false
- c) false e true
- d) false e false



Lembre-se que == não testa o tipo, fazendo uma coerção para que os dois tipos sejam iguais. Assim, o primeiro console retorna true pois ele irá testar apenas se os valores são

iguais, sem considerar os tipos, já o segundo considera tipos o que retorna false pois os tipos são diferentes.

Letra b.

017. (FCPC/UFC/ANALISTA DE TI/ÁREA: ARQUITETURA E DESENVOLVIMENTO DE SISTEMAS – BACK-END/2025) Sobre as características do NodeJS, marque a alternativa correta.

- a) O NodeJS foi construído para ser utilizado em servidores Linux e Windows, não fornecendo suporte ao sistema operacional MacOS.
- b) Por se basear na linguagem JavaScript, somente é possível criar aplicações Web com o NodeJS. Entretanto, fornece a escalabilidade necessária em sistemas Web.
- c) O NodeJS é um ambiente de execução JavaScript que roda do lado servidor da comunicação, possibilitando a geração de páginas dinâmicas antes de serem enviadas aos clientes.
- d) Uma das características inovadoras do NodeJS é a capacidade de possibilitar uma comunicação Web síncrona, aguardando finalizar o atendimento de uma solicitação para tratar a seguinte.



Vamos analisar as assertivas:

- a) Errada. O NodeJS é multiplataforma e suporta Linux, Windows e MacOS. Assim, existem versões do NodeJS para MAC.
- b) Errada. O NodeJS permite a criação de quaisquer tipos de apps, não apenas WEB.
- c) Certa. O NodeJS permite executar JavaScript no servidor, o que possibilita a geração de conteúdo dinâmico antes de ser enviado para o navegador do cliente.
- d) Errada. O NodeJS tem como característica fundamental a ASSINCRONICIDADE (assim como o JS, de forma geral). Pode lidar com várias solicitações simultaneamente sem esperar que cada uma seja concluída antes de começar a próxima.

Letra c.

018. (FCPC/UFC/TÉCNICO DE TI/ÁREA: DESENVOLVIMENTO DE SOFTWARE – BACK-END/2025) Marque a alternativa que contém o nome do recurso existente na linguagem JavaScript que possibilita o alinhamento da declaração de funções, onde a função mais interna possui acesso aos recursos definidos na função externa?

- a) Sorted.
- b) Closures.
- c) Promises.
- d) WeakMap.



- a) Errada. Sorted não é um recurso fundamental da linguagem JavaScript. A ordenação de arrays em JavaScript é feita com o método `sort()`, mas não há um recurso chamado Sorted.
- b) Certa. Closures são funções que “lembram” do ambiente em que foram criadas, mesmo após esse ambiente ter terminado de executar. Isso significa que uma função interna pode acessar variáveis e outros recursos da função externa em que foi definida.

Em JavaScript, uma closure é a combinação de uma função com o ambiente léxico em que essa função foi declarada. Isso significa que uma função interna (definida dentro de outra função) tem acesso às variáveis da função externa, mesmo depois que a função externa já foi executada.

- c) Errada. Promises são objetos que representam o resultado eventual de uma operação assíncrona. Eles são usados para lidar com operações que levam tempo para serem concluídas, como requisições a um servidor, mas não estão diretamente relacionados ao alinhamento de declaração de funções ou ao acesso a recursos de funções externas.

- d) Errada. WeakMap é uma coleção de pares chave-valor em que as chaves são objetos e os valores podem ser de qualquer tipo. As chaves são fracamente referenciadas, o que significa que, se não houver outras referências ao objeto chave, ele pode ser coletado pelo garbage collector. WeakMap não está diretamente relacionado ao alinhamento **de declaração de funções ou ao acesso a recursos de funções externas.**

Letra b.

019. (CESPE-CEBRASPE/PC-DF/GESTOR DE APOIO ÀS ATIVIDADES POLICIAIS CIVIS/ESPECIALIDADE: ANALISTA DE INFORMÁTICA: DESENVOLVIMENTO DE SISTEMAS/2025) Julgue o próximo item, a respeito de desenvolvimento de sistemas.

Na execução do trecho de código a seguir, escrito em JavaScript, o resultado lógico da operação `x == 7` será falso.



```
<script>
let x = "7";
document.getElementById("teste").innerHTML =
(x == 7);
</script>
```

Errado, lembre-se o `==` não testa tipo. Será feita uma coerção para que “7” e 7 sejam do mesmo tipo, ou seja, o resultado será verdadeiro

Errado.

020. (IMPARH/CGM DE FORTALEZA-CE/AUDITOR DE CONTROLE INTERNO – ÁREA 2 (CIÊNCIAS DA COMPUTAÇÃO)/2025) Considere as afirmações abaixo e marque a opção correta.

I – O comando break pode ser usado para interromper um loop for ou while em JavaScript. Esse comando permite que o loop termine antes que sua condição final seja atingida.

PORQUE

II – Quando o break é utilizado em um loop, ele encerra a iteração atual e passa para a próxima, mantendo o loop em execução.

- a) As duas afirmações são verdadeiras e a segunda justifica a primeira.
- b) As duas afirmações são verdadeiras, mas a segunda não justifica a primeira.
- c) A primeira afirmação é verdadeira e a segunda é falsa.
- d) A primeira afirmação é falsa e a segunda é verdadeira.



A primeira alternativa é verdadeira já que é exatamente o que o break faz. O comando break em JavaScript é usado para sair de um loop (como for ou while) antes que a condição de término do loop seja satisfeita.

Já a segunda é falsa uma vez que essa é a função do continue. O comando break encerra completamente o loop, e não apenas a iteração atual. A execução do programa continua a partir da próxima instrução após o loop.

Letra c.

021. (INSTITUTO ACESSO/CÂMARA DE MANAUS-AM/ANALISTA DE SEGURANÇA DA INFORMAÇÃO/2024) TypeScript é compilado para JavaScript, o que significa que o código TypeScript é transformado em código JavaScript que pode ser executado em qualquer navegador ou ambiente que suporte JavaScript. O código TypeScript ao ser compilado, se tornaria, por exemplo, a isso em JavaScript:

```
function saudacao(nome: string) {
  return `Olá, ${nome}!`;
}
```

```
console.log(saudacao("Mundo TypeScript"));
```

- a) `function saudacao(nome:string) { return "Olá, " + nome + "!"; } console.log(saudacao("Mundo TypeScript"));`
- b) `function saudacao(string) { return "Olá, " + nome + "!"; } console.log(saudacao("Mundo TypeScript"));`
- c) `function saudacao(nome) { return "Olá, " + nome + "!"; } console.log(saudacao("Mundo TypeScript"));`

d) `function saudacao(nome) { return "Olá, " + nome + "!"; } console.log(saudação("Mundo TypeScript"));`

e) `function saudacao(nome) { return `Olá, ${nome}!`; } console.log(saudacao("Mundo TypeScript"));`



Basicamente, o que se deve fazer para resolver essa questão é retirar todos os tipos. Como o único tipo existente é o parâmetro do tipo string, essa será a única mudança.

Letra e.

022. (INSTITUTO ACESSO/CÂMARA DE MANAUS-AM/ANALISTA DE SISTEMAS/2024) Webpack e NPM são ferramentas essenciais no desenvolvimento de aplicações modernas em JavaScript. Enquanto NPM é usado para gerenciar pacotes e dependências, o Webpack ajuda a organizar e empacotar os arquivos. Qual das alternativas abaixo representa corretamente uma funcionalidade do Webpack?

a) Webpack permite modularizar e empacotar arquivos de código, incluindo CSS e imagens, facilitando o carregamento otimizado no navegador.

b) Webpack é uma ferramenta de gerenciamento de pacotes que instala dependências no projeto, como bibliotecas externas e frameworks.

Webpack é responsável pela execução do código JavaScript no lado do servidor, substituindo o Node.js.

c) Webpack gera um arquivo package.json que armazena as dependências do projeto e versões específicas.

d) Webpack serve para compilar apenas código JavaScript, sem suportar a inclusão de arquivos CSS ou imagens.



a) Certa. A letra A explica exatamente o que é o Webpack, um module bundler que pega todos os arquivos (JavaScript, CSS, imagens, etc.) que sua aplicação precisa e os transforma em um ou mais arquivos otimizados para serem servidos no navegador.

b) Errada. Na Letra B descreve-se o NPM. O gerenciamento de pacotes é a função do NPM (Node Package Manager) ou Yarn, não do Webpack.

c) Errada. Não faz sentido pois o Webpack não substitui o NodeJS.

d) Errada. Quem gera o package.json é o NPM através do parâmetro init.

e) Errada. Há plugins para o webpack que permitem que ele inclua o CSS no bundle.

Letra a.

023. (CESPE-CEBRASPE/TSE/TÉCNICO JUDICIÁRIO/ÁREA: APOIO ESPECIALIZADO – ESPECIALIDADE: PROGRAMAÇÃO DE SISTEMAS/2024) Julgue o próximo item, relativos a linguagens e tecnologias de programação.

O código a seguir, escrito em HTML e JavaScript, ao ser executado, apresentará 2 como resultado.

```
<html>
<body>
<p id="cpnuje"></p>
<script>
for (let i = 0; i < 10; ++i) {
if (i === 3) { break; }
document.getElementById("cpnuje").innerHTML
= i;
}
</script>
</body>
</html>
```



O código irá parar o loop (break) quando i for igual a 3, assim, a última vez que ele irá mudar o valor do parágrafo (p) será com i igual a 2.

Certo.

024. (FUNDATEC/SAAE/VIÇOSA-MG/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2024) Analise o código HTML abaixo, o qual contém programação na linguagem JavaScript.

```
<html>
<body>
<h2>Minha primeira Web Page</h2>
<button type="button" name="Tente" onclick="document.write('>Clicou>')">Tente</button>
</body>
</html>
```

O que será exibido ao clicar no botão “Tente”?

- a) >Clicou>
- b) >Clicou>Tente
- c) Tente
- d) Minha primeira Web Page>Clicou>
- e) Minha primeira Web Page>Clicou>Tente



A resposta correta é a letra A. Lembrando que document.write já não consegue ser executada em alguns navegadores por segurança. Para saber mais:

<https://developer.mozilla.org/en-US/docs/Web/API/Document/write>

Letra a.

025. (FUNDATEC/IF-AP/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2024) “Na linguagem JavaScript, o texto entre _____ e o final de uma linha é tratado como comentário”.

Assinale a alternativa que preenche corretamente a lacuna do trecho acima.

- a) //
- b) \\
- c) %%
- d) --
- e) ??



Os **comentários** são uma parte essencial de qualquer linguagem de programação, incluindo JavaScript. Eles **permitem que os desenvolvedores adicionem anotações, explicações ou desativem trechos de código sem que isso afete a execução do programa**.

Em Javascript existem dois tipos principais de comentários:

- **Comentários de Uma Linha**

Sintaxe: usa duas barras (//) no início da linha. Tudo após as barras, até o final da linha, é considerado um comentário.

- **Comentários de Várias Linhas**

Sintaxe: inicia com /* e termina com */. Todo o conteúdo entre esses marcadores é considerado um comentário, mesmo que ocupe várias linhas.

Letra a.

026. (FGV/TJ-RR/ANALISTA JUDICIÁRIO/DESENVOLVIMENTO DE SISTEMAS/2024) Em muitas linguagens de programação, o bloco try-catch é uma estrutura fundamental para o tratamento de exceções.

No Javascript, para tratar múltiplas exceções dentro de um mesmo bloco try-catch, podemos

- a) utilizar múltiplos blocos catch após o mesmo bloco try, cada um para uma exceção específica.
- b) especificar os diferentes tipos de exceção na cláusula catch, separados por vírgulas.
- c) utilizar o bloco finally para capturar exceções adicionais não tratadas pelo bloco catch.
- d) usar um único bloco catch e, dentro dele, diferenciar as exceções usando estruturas condicionais como if ou switch.
- e) declarar os tipos de exceção diretamente no bloco try usando parâmetros específicos.



Em Javascript não há como haver mais de um catch para um try, dessa forma a única forma de fazer é a letra D. Veja em detalhes aqui:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/try...catch#Conditional_catch_clauses

Letra d.

027. (UFES/UFES/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2024) Em um projeto de desenvolvimento *web*, é necessário que, ao clicar um botão, o texto de um elemento *div* seja atualizado para “Texto Atualizado”.

Considerando as boas práticas em JavaScript, a abordagem adequada para implementar a funcionalidade de atualizar o texto conforme descrito é:

- a) Alterar o atributo *value* do *div* para “Texto Atualizado” e definir um onclick diretamente no elemento *button* para executar a alteração.
- b) Atualizar o texto do *div* diretamente na função de carregamento da página, sem adicionar um manipulador de eventos ao botão, para definir o texto inicial.
- c) Modificar o estilo do *div* para mostrar o texto “Texto Atualizado” e adicionar um manipulador de eventos *onClick* ao botão, utilizando o estilo para esconder e exibir o texto.
- d) Usar a propriedade *textContent* para modificar o texto do *div* e configurar a alteração do texto utilizando uma função anônima diretamente na propriedade *onclick* do botão.
- e) Utilizar a propriedade *innerHTML* para alterar o conteúdo do *div* e adicionar um *eventListener* ao botão, criando uma função de manipulador de eventos separada, para garantir a modularidade e a manutenção do código.



A função *innerHTML* é a ideal para fazer isso pois fará com que o valor colocado seja propriamente tratado como HTML. Quanto à forma de fazer, adicionar um *eventListener* ao botão é a forma mais correta no Vanilla JS (Javascript puro).

innerHTML: A propriedade *innerHTML* é usada para definir ou obter o conteúdo HTML de um elemento. É adequada para atualizar o texto de um *div*, pois permite substituir o conteúdo existente por um novo.

eventListener: Adicionar um *eventListener* ao botão é a maneira correta de lidar com eventos em JavaScript. Isso separa a lógica de manipulação de eventos do HTML, promovendo um código mais limpo e organizado.

Função de Manipulador de Eventos Separada: Criar uma função separada para lidar com o evento de clique do botão promove a modularidade do código. Isso significa que a função pode ser reutilizada em outros lugares, e o código se torna mais fácil de entender e manter.

Letra e.

028. (FUNDATEC/PREFEITURA DE CRUZ ALTA – RS/ANALISTA DE SISTEMAS/2024) Na linguagem JavaScript, qual das instruções abaixo é utilizada para interromper a execução de uma função e retornar um valor para a instrução que invocou essa função?

- a) stop
- b) return

- c) break
- d) go back
- e) exit



A resposta correta é a B pois o return é responsável por interromper a execução da função e retornar um valor. Importante lembrar que o return pode ser utilizado SEM RETORNAR NADA EXPLICITAMENTE.

Letra b

029. (QUADRIX/CRT-MG/GESTOR DE TI/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Uma das desvantagens do CSS3 é que ele não permite especificar a quantidade de espaço entre as linhas, o espaçamento entre caracteres ou as margens da página.



O CSS3 permite especificar:

- Espaçamento entre linhas com a propriedade line-height.
- Espaçamento entre caracteres com letter-spacing.
- Margens com margin.

Essas propriedades são essenciais no design de páginas web e estão disponíveis desde versões anteriores do CSS, não apenas no CSS3.

Errado.

030. (QUADRIX/CRT-MG/GESTOR DE TECNOLOGIA DA INFORMAÇÃO/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Os atributos globais são aqueles que podem ser usados somente com os elementos novos da HTML 5, e não com todos os seus elementos.



Os **atributos globais** no HTML5 são aqueles que podem ser usados em **qualquer elemento HTML**, não apenas nos novos elementos introduzidos pelo HTML5. Exemplos de atributos globais incluem id, class, style, data-*, hidden, tabindex, entre outros. Eles são aplicáveis a praticamente todas as tags HTML, independentemente de serem novas ou antigas.

Errado.

031. (QUADRIX/CRT-MG/GESTOR DE TECNOLOGIA DA INFORMAÇÃO/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Em HTML5, o atributo *src* fornece as especificidades da imagem que o programador deseja incluir.



src diz de qual origem (source) a imagem será carregada.

Errado.

032. (QUADRIX/CRT-MG/GESTOR DE TECNOLOGIA DA INFORMAÇÃO/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Em JavaScript, a instrução *do...while* repete a execução de um bloco de código, enquanto uma condição for satisfeita, mas o executa pelo menos uma vez, mesmo que a condição nunca seja satisfeita.



Essa informação está CERTA uma vez que a primeira vez que a condição será testada é depois que o *while* foi já executado uma vez

Certo.

033. (QUADRIX/CRT-MG/GESTOR DE TECNOLOGIA DA INFORMAÇÃO/2022) A respeito da linguagem JavaScript, do HTML 5 e do CSS3, julgue o item.

Na linguagem de programação JavaScript, o evento *blur* é executado sempre que um documento é abandonado.



Ela é executada sempre que um elemento que pode ser selecionável (botão, input etc) perde o foco.

Errado.

034. (IF-SP/IF-SP/PROFESSOR EBTT – INFORMÁTICA/2024) Frameworks de desenvolvimento WEB possuem bibliotecas, padrões de componentes e de design que podem ser utilizados pelos desenvolvedores para facilitar e acelerar a criação de aplicativos. Marque a alternativa correta sobre os frameworks apresentados a seguir e a linguagem na qual eles são baseados.

- a) React é um framework de desenvolvimento WEB baseado em JavaScript.
- b) Django é um framework de desenvolvimento WEB baseado em PHP.
- c) Laravel é um framework de desenvolvimento WEB baseado em JavaScript.
- d) AngularJS é um framework de desenvolvimento WEB baseado em Python.



Essa questão foge um pouco do nosso tema, mas vale a pena saber, não é? Além disso, com o conhecimento em React que vc adquiriu, é possível responde-la pois a primeira opção já é a verdadeira. De resto...

Django é baseado em Python

Laravel é baseado em PHP

AngularJS é baseado em JS

Letra a.

035. (FGV/SEDUC-SP/PROFESSOR DE ENSINO FUNDAMENTAL E MÉDIO (EDUCAÇÃO PROFISSIONAL) – TECNOLOGIA DA INFORMAÇÃO/2024) Em JavaScript, a palavra-chave _____ é usada para declarar variáveis cujo valor pode mudar no escopo do bloco em que foram definidas, enquanto a palavra-chave _____ é usada para variáveis imutáveis. Já o método _____ adiciona elementos ao final de um array e a estrutura _____ permite a repetição de um bloco de código enquanto uma condição for verdadeira, já o objeto _____ fornece métodos para manipulação de números e cálculos matemáticos.

Em sequência, as palavras-chave que completam corretamente essas lacunas são:

- a) const, let, while, push, Math.
- b) while, push, Math, const, let.
- c) let, const, push, while, Math.
- d) Math, const, let, while, push.
- e) while, Math, const, push, let.



Vamos preencher as lacunas com as palavras-chave corretas:

“Em JavaScript, a palavra-chave let é usada para declarar variáveis cujo valor pode mudar no escopo do bloco em que foram definidas, enquanto a palavra-chave const é usada para variáveis imutáveis.”

let permite declarar variáveis que podem ser reatribuídas.

const declara variáveis cujo valor não pode ser alterado após a atribuição inicial.

“Já o método push adiciona elementos ao final de um array...”

push() é um método de array que adiciona um ou mais elementos ao final do array e retorna o novo comprimento do array.

“...e a estrutura while permite a repetição de um bloco de código enquanto uma condição for verdadeira...”

while é uma estrutura de controle de fluxo que executa um bloco de código repetidamente enquanto uma condição especificada for avaliada como verdadeira.

“..já o objeto Math fornece métodos para manipulação de números e cálculos matemáticos.” Math é um objeto embutido em JavaScript que possui propriedades e métodos para constantes e funções matemáticas.

Portanto, a sequência correta é: let, const, push, while, Math.

Letra c.

036. (PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Ao trabalhar com padrões web em JavaScript, você deseja criar um script que seja executado apenas após a carga completa da página. Qual evento garante que o script seja executado no momento adequado?

- a) 'DOMContentLoaded'
- b) 'load'
- c) 'onload'
- d) 'documentReady'



A melhor forma de fazer o que se pede é com o DomContentLoaded. Para mais informações detalhadas, vá em:

https://developer.mozilla.org/en-US/docs/Web/API/Document/DOMContentLoaded_event

Letra a.

037. (PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Na escolha de um framework JavaScript para um novo projeto front-end, é essencial considerar as características únicas que cada um oferece para atender às necessidades específicas do projeto. Ao analisar opções para um projeto front-end, qual característica distingue o Svelte de outros frameworks JavaScript, como Angular, React ou Vue?

- a) Virtual DOM.
- b) Compilação em tempo de construção.
- c) Two-way databinding.
- d) Uso de JSX.



O Swelt compila o código, dessa forma, não há um Runtime no navegador como o React, Vue e Angular tem, dessa forma a opção: Compilação em tempo de construção é a correta

Letra b.

038. (UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Ao implementar um novo recurso em um site com JavaScript, é preciso armazenar uma coleção de valores. Qual estrutura de dados é mais adequada para esse propósito?

- a) String.
- b) Boolean.
- c) Array.
- d) Character.



O Array permite armazenar uma coleção de valores em JS. Há outras formas de se fazer isso, mas não existem essas formas como uma opção na questão

Letra c.

039. (PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS/ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Durante a codificação de um script em JavaScript, foi necessário declarar uma variável cujo valor pode mudar ao longo do tempo dentro de um bloco específico. Qual palavra-chave utilizar nesse caso?

- a) 'const'
- b) 'let'
- c) 'var'
- d) 'static'



Let e var permitem que a variável mude de valor. Mas apenas let permite mudar apenas dentro do bloco.

Letra b.

040. (CESPE/CEBRASPE/TCE-AC/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO – ÁREA: PROJETOS DE TI/2024) Em relação ao desenvolvimento de sistemas web, julgue o próximo item. Nas aplicações SPA (single page application) que utilizam AJAX, cada interação do usuário resulta em um recarregamento completo da página, o que garante que todas as partes da interface sejam atualizadas simultaneamente.



A questão mostra exatamente o **OPOSTO** da SPA, em que a app é carregada apenas uma vez, sendo que depois, apenas chamadas AJAX mudam partes do documento SEM recarregá-lo.

Errado.

041. (CESPE-CEBRASPE/TCE-AC/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO – ÁREA: PROJETOS DE TI/2024) Acerca de desenvolvimento web e mobile, julgue o item seguinte. O objetivo do JavaScript é deixar mais dinâmicas as aplicações web, de maneira que o usuário possa interagir e alterar o conteúdo da página.



Essa foi exatamente a motivação da criação do JS.

Certo.

042. (FGV/TRF – 1ª REGIÃO/TÉCNICO JUDICIÁRIO – ÁREA ADMINISTRATIVA – ESPECIALIDADE: DESENVOLVIMENTO DE SISTEMAS DE INFORMAÇÃO/2024) O analista Cléber está implementando um script para automatizar o build de uma aplicação apoiada pelo npm. Para se certificar de que, a cada novo build, o npm reinstalará todas as dependências da aplicação sem alterar o arquivo package.json, Cléber empregou no script de automação o recurso de instalação limpa do npm.

Para solicitar ao npm uma instalação limpa do projeto, Cléber utilizou no script o comando npm:

- a) ci;
- b) dedupe;
- c) shrinkwrap;
- d) install --clear;
- e) install --erase.



O npm ci faz a instalação limpa do projeto, letra A

Letra a.

043. (FGV/BBTS/2023) Os tipos de dados suportados pela linguagem ECMAScript (versão 2021) são

- a) Null, Boolean, Binary, Symbol, Real e Integers.
- b) Null, Boolean, Varchar, Symbol, Real, Integers e Object.
- c) Undefined, Null, Boolean, String, Symbol, Number, BigInt e Object.
- d) Undefined, Null, Boolean, Varchar, Symbol, BigInt, Real, Money e Polimorphic.
- e) Complex, Null, Boolean, Binary, Varchar, LongInt, Real, Money, Object e Polimorphic



Na especificação ECMAScript 2021 (também conhecida como ES12), os tipos de dados suportados são:

- Undefined: representa uma variável que não foi atribuída a um valor.
- Null: representa um valor intencionalmente vazio ou inexistente.
- Boolean: representa um valor verdadeiro (true) ou falso (false).
- String: representa uma sequência de caracteres (texto).
- Symbol: um tipo de dado único e imutável, frequentemente usado para identificar propriedades de objeto de forma única.
- Number: representa números inteiros e de ponto flutuante.
- BigInt: representa inteiros de precisão arbitrária, permitindo a representação de números maiores do que Number pode suportar.
- Object: um tipo de dado complexo que pode conter coleções de valores e mais tipos complexos.

As demais opções listam tipos que **não** são parte da especificação ECMAScript, como Varchar, Binary, Real, Money, Complex, LongInt, e Polimorphic, que não são tipos de dados em JavaScript.

Portanto, a opção que corretamente enumera os tipos de dados suportados pelo ECMAScript 2021 é a C.

Letra c.

044. (FGV/TJ RN/2023) No contexto da linguagem JavaScript, analise as afirmativas sobre variáveis declaradas como const.

I – Devem ter seus valores atribuídos na declaração.

II – Podem ter seus valores alterados depois de declaradas.

III – Têm sempre escopo global.

Está correto o que se afirma em:

- somente II e III;
- somente I;
- somente II;
- somente III;
- I, II e III.



Vamos analisar as afirmativas:

I – Certa. Em JavaScript, quando você declara uma variável usando const, é obrigatório inicializá-la com algum valor na própria declaração. Por exemplo, const x = 10; é válido, enquanto const x; não é. **ATENÇÃO:** Apesar de seus valores terem que ser atribuídos na declaração, é possível modificar valores dentro de um objeto ou array definido como const.

Vamos lembrar um exemplo:

```
const A = {X:1};
```

a) `X = 2;` // Isso é válido e está mudando o valor de X e não de A

```
const A = [1,2];
```

`A[2] = 3;` // Também é válido pois está mudando o valor do segundo item

II – Errada. Uma variável declarada com `const` não pode ter seu valor reassignado. No entanto, se o valor for um objeto ou um array, o conteúdo interno do objeto ou array pode ser modificado, pois a constante apenas protege a ligação do identificador ao valor.

III – Errada. Variáveis declaradas com `const` têm escopo de bloco, o que significa que elas só são acessíveis dentro do bloco em que foram definidas, semelhante ao `let`. Elas não são automaticamente globais. A única variável com escopo global é a `var`.

Portanto, a única afirmação correta é a afirmativa I.

Letra b.

045. (FGV/SEF MG/2023) Considere as seguintes expressões usando a linguagem javascript:

```
1 == '1' e 1 === '1'
```

Os resultados são, respectivamente,

a) 1 e 1

b) true e true

c) '1' e '1'

d) false e false

e) true e false



```
1 == '1':
```

O operador `==` é o operador de igualdade fraca em JavaScript, que compara dois valores para igualdade após converter (coerção) seus tipos se necessário.

Neste caso, `1 == '1'` é comparado após a conversão de tipo. A string `'1'` é convertida para o número 1, resultando em `1 == 1`, que é `true`.

```
1 === '1':
```

O operador `===` é o operador de identidade estrita, que compara tanto o valor quanto o tipo dos operandos, sem fazer conversões de tipo.

Neste caso, `1 === '1'` compara um número (1) com uma string (`'1'`). Como os tipos são diferentes, a expressão avalia para `false`.

Portanto, os resultados das expressões são, respectivamente, `true` e `false`.

Letra e.

046. (FGV/TJ RJ/2023) No contexto de um script JavaScript numa página Web, considere o trecho a seguir.

```
<script>
function teste(x) {
return (x / 2);
}
alert (teste);
</script>
```

É correto afirmar sobre esse script que:

- a) há um erro de sintaxe que impede sua execução;
- b) quando executado, exibe a definição da função teste;
- c) quando executado, exibe o valor 0;
- d) quando executado, exibe o valor undefined;
- e) quando executado, o comando alert não é acionado.



O trecho de código define uma função teste(x) que retorna o valor de $x / 2$.

A instrução alert(teste); passa a referência da função teste — sem parênteses, o que significa que não está chamando a função, mas sim referenciando-a.

Quando você passa uma função diretamente para o alert, sem executá-la (sem usar ()), o JavaScript converte a função em uma string para exibição. A string resultante é a definição da função.

Portanto, o script exibirá a definição da função teste, e não o resultado de alguma execução ou operação dentro da função.

Letra b.

047. (FGV/CÂMARA DOS DEPUTADOS/2023) Analise o pretenso código JavaScript a seguir.

```
const xpto = (a, b) => a + b;
```

Sobre esse trecho, é correto afirmar que esse código:

- a) é válido, porque define uma função que retorna a soma de dois números fornecidos como parâmetros;
- b) não compila devido à ausência de um par dos delimitadores "{" e "}";
- c) não compila porque o símbolo "=>" não é permitido na definição de uma função JavaScript;
- d) não compila porque o símbolo "=>" não é permitido numa declaração const;
- e) pode ser compilado, mas gera erro em tempo de execução quando xpto for invocada.



A alternativa correta é:

- a) é válido, porque define uma função que retorna a soma de dois números fornecidos como parâmetros;

- O trecho de código usa a sintaxe das arrow functions em JavaScript, introduzidas na ECMAScript 2015 (ES6). As arrow functions são uma maneira concisa de escrever funções em JavaScript.
- `const xpto = (a, b) => a + b;` define uma função chamada xpto que aceita dois parâmetros, a e b.
- A expressão `a + b` é o corpo da função, e ela retornará implicitamente o resultado da soma `a + b`.
- Não é necessário usar `{ }` para delimitar o corpo da função quando ele consiste apenas de uma única expressão, como é o caso aqui.

Portanto, o código é válido e funcionará corretamente, retornando a soma de dois números fornecidos como argumentos quando a função xpto for chamada.

Letra a.

048. (FCC/TRT 20/2016) Em JavaScript, a função do método `indexOf()` é

- procurar um valor dentro de uma string e retornar a quantidade de ocorrências desse valor, se encontrado.
- retornar o índice da última ocorrência do texto especificado em uma string.
- trair um texto de uma string e retornar a string com o texto extraído em uma nova string.
- trocar um valor especificado por espaço em branco na string.
- retornar o índice da primeira ocorrência de um texto específico em uma string.



A alternativa correta é:

- retornar o índice da primeira ocorrência de um texto específico em uma string.
 - O método `indexOf()` em JavaScript é usado para buscar a posição da primeira ocorrência de um valor específico dentro de uma string.
 - Se o texto especificado for encontrado, `indexOf()` retorna o índice da primeira ocorrência. O índice começa em 0. Se o texto não for encontrado, o método retorna -1.
 - Exemplo:

```
const str = "Hello, world!";
const index = str.indexOf("world");
console.log(index); // Output: 7
```

Este método não conta o número de ocorrências (como a opção a sugere), não retorna o índice da última ocorrência (como na opção b), não extrai texto (como na opção c), e não substitui valores por espaço (como na opção d).

ATENÇÃO

Existe também em Javascript o método `indexOf` em um `ARRAY`. Nesse caso ele retornará o primeiro elemento do array que corresponde ao parâmetro. Segue um link para o detalhamento desse método: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/indexOf

No nosso entendimento a questão se tornou um pouco confusa pois não mencionou essa forma, mas temos que nos preparar para isso também!

Letra e.

049. (FGV/SEF MG/2023) Analise o código Javascript a seguir.

```
let a = {x: 1};
let b = {x: 2};
let c = {x: 3};
let arr = [a, b, c];
arr.forEach((obj) => {
  obj.x += 1;
});
console.log(a.x, b.x, c.x);
```

O resultado exibido ao rodar esse código será

- a) 1 2 3
- b) 2 3 4
- c) 4 5 6
- d) [1, 2, 3]
- e) [2, 3, 4]



O resultado exibido ao rodar esse código será:

- b) 2 3 4

Explicação:

1. O código define três objetos `a`, `b`, e `c` com uma propriedade `x` inicializada para 1, 2, e 3, respectivamente.
2. Esses objetos são então armazenados em um array chamado `arr`.
3. O método `forEach()` é chamado no array e itera sobre cada objeto usando uma função de seta `(obj) => { obj.x += 1; }`.
 - Para cada objeto em `arr`, a propriedade `x` é incrementada em 1.
4. Finalmente, `console.log(a.x, b.x, c.x);` exibe os valores das propriedades `x` dos objetos `a`, `b` e `c`.

Portanto, após a execução do `forEach`, os valores das propriedades `x` dos objetos `a`, `b`, e `c` serão 2, 3, e 4, respectivamente.

Letra b.

050. (FGV/PC AM/2022) Considere o trecho JavaScript exibido a seguir.

```
const a = [1,2,3,4,5];
const b = a.map(xpto);
alert(b);
function xpto (x, y) {
  return x * y;
}
```

Assinale o conteúdo exibido na execução do script acima.

- a) 0,2,6,12,20
- b) 1,2,3,4,5
- c) 1,3,5,7,9
- d) 1,4,9,16,25
- e) 2,4,6,8,10



A alternativa correta é:

a) 0,2,6,12,20

Explicação:

- O método `map()` cria um novo array com os resultados da chamada de uma função para cada elemento do array original, `a`.
- A função `xpto` é definida como `function xpto (x, y) { return x * y; }`. Quando usada com `map()`, `x` representa o elemento do array, e `y` representa o índice do elemento.

Assim, em cada iteração, a função `xpto` multiplica o valor do elemento pelo seu índice correspondente no array `a`:

- Para o primeiro elemento 1 (índice 0): $1 * 0 = 0$

- Para o segundo elemento 2 (índice 1): $2 * 1 = 2$
- Para o terceiro elemento 3 (índice 2): $3 * 2 = 6$
- Para o quarto elemento 4 (índice 3): $4 * 3 = 12$
- Para o quinto elemento 5 (índice 4): $5 * 4 = 20$

Portanto, o array `b` resultante é `[0, 2, 6, 12, 20]`, que é a sequência exibida pelo `alert`.

Letra a.

REFERÊNCIAS

<https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide>

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Loops_and_iteration

<https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Functions>

<https://www.alura.com.br/artigos/manipulacao-de-array-com-map-filter-e-reduce?srsId=AfmBOopX3Td4g9axxy4edl40dHMXeTuo5Fvaw1lIOhLlO9xtSTykliQ7>

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Promise

https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API/Using_Fetch

<https://developer.mozilla.org/pt-BR/docs/Web/API/XMLHttpRequest>

Abra



caminhos



crie

futuros

gran.com.br

